University of Economics, Prague

Faculty of Informatics and Statistics

BACHELOR THESIS

Prague, December 2024 Vadym Tatarinov

University of Economics, Prague

Faculty of Informatics and Statistics

Title of the bachelor thesis:

Image classification using machine learning methods

Author: **Vadym Tatarinov**

Department: **Department of Information Technology**

Study Field: **Applied Informatics**

Supervised by: **Ing. Lukáš Veverka**

Declaration of Authorship

Hereby I declare that the work presented in this bachelor thesis titled *Image classification using machine learning methods* is my own except for the cited parts. The literature and supporting materials are mentioned in the bibliography on pages 73-76.

December 8, 2024, Prague .....

Vadym Tatarinov

Abstract

Recent advancements in deep learning have changed computer vision, evolving from traditional methods like Support Vector Machines (SVM) to Convolutional Neural Networks (CNN) in 2012, and more recently to Vision Transformers (ViT) in 2021. Simultaneously, improvements in computational power and the exponential growth of internet data have further driven this progress. This work uses these developments to build a computer vision model for classifying damaged and undamaged cars. Specifically, the study compares pre-trained convolutional neural networks, such as ResNet and ConvNeXt, to find the best-performing architecture for this task. Key contributions of this work include an analysis of the impact of data quantity and quality on model performance, the balance between computational resources and accuracy, and an exploration of hyperparameter optimization strategies for this application. The model's performance was evaluated using metrics like accuracy, model size, and F1 score, with ConvNeXt achieving a maximum accuracy of 95.4%, demonstrating its effectiveness in binary classification for this task. The final model from this study could be applied to online platforms or auctions for selling used cars, potentially enhancing ad conversion rates, improving user experience, and reducing fraud associated with selling damaged vehicles.

Keywords: Computer vision, Image recognition, Image classification, Pytorch, Deep learning, Fine-tuning, Transfer learning, Car damage detection

Abstrakt

Nedávné pokroky v oblasti hlubokého učení změnily počítačové vidění, které se vyvinulo z tradičních metod, jako je Support Vector Machines (SVM), na konvoluční neuronové sítě (CNN) v roce 2012 a nedávno na Vision Transformers (ViT) v roce 2021. Současně zlepšení výpočetního výkonu a exponenciální nárůst internetových dat tento pokrok dále podpořily. Cílem této práce je využít tento vývoj k vytvoření modelu počítačového vidění pro klasifikaci poškozených a nepoškozených automobilů. Konkrétně studie porovnává různé předtrénované konvoluční neuronové sítě, jako jsou ResNet a ConvNeXt, s cílem najít pro tuto úlohu nejvýkonnější architekturu. Mezi hlavní přínosy této práce patří analýza vlivu množství a kvality dat na výkonnost modelu, rovnováha mezi výpočetními zdroji a přesností a zkoumání strategií optimalizace hyperparametrů pro tuto aplikaci. Výkonnost modelu byla hodnocena pomocí ukazatelů, jako je přesnost, velikost modelu a skóre F1, přičemž ConvNeXt dosáhl maximální přesnosti 95.4%, což dokazuje jeho účinnost při binární klasifikaci pro tuto úlohu. Konečný model z této studie by mohl být použit na online platformách nebo aukcích pro prodej ojetých automobilů, což by mohlo zvýšit míru konverze reklamy, zlepšit uživatelský komfort a omezit podvody spojené s prodejem poškozených vozidel.

Klíčová slova: Počítačové vidění, Rozpoznávání obrazu, Klasifikace obrazu, Pytorch, Hluboké učení, Fine-tuning, Transfer learning, Detekce poškození automobilu

Contents

Introduction

The increasing popularity of online marketplaces for buying and selling vehicles has created a growing demand for automated tools to assess vehicle conditions through images. Accurate classification of vehicle damage is crucial for building trust and efficiency in these platforms, yet existing models, such as those available on Kaggle, often fall short, achieving accuracies around 90% and suffering from reproducibility issues. This limitation motivated the development of a custom model trained on a dataset prepared with modern deep learning methods.

The goal of this study is to train and compare pre-trained models for the classification of damaged and undamaged vehicles, utilizing heuristic methods and state-of-the-art neural network architectures. Key information sources include preprints from the arXiv archive, which discuss the principles and effectiveness of heuristics, as well as Stanford University's deep learning course.

The data for this project was collected from open datasets on Kaggle, with the majority of images scraped from various online platforms for selling used vehicles. The final model architecture was designed to operate on both CPU[1] and GPU[2], enabling flexible deployment options. Additionally, a simple API was created using FastAPI, and a graphical user interface (GUI) was developed with Streamlit. This web application allows users to upload images and test the model's performance in real time, making the solution accessible to a wider audience. The app, along with an API, is deployed on a private VPS[3] and can be accessed online for demonstration purposes[4].

For this project, the PyTorch library and custom Python scripts were used to prepare data, optimize hyperparameters, and construct model architectures. The inclusion of the Streamlit app emphasizes the research's practical utility, bridging the gap between academic exploration and real-world deployment.

[1]CPU - Central processing unit
[2]GPU - Graphics processing unit
[3]VPS - Virtual private server
[4]Streamlit web application is available on https://eucalytics.uk

1 Literature review

I used the research of other authors as the basis for my study in order to correct their shortcomings and enchament them with new neural network model architectures like ConvNext. Many works on this topic have one drawback in common - the lack of training data, for example Car Damage Detection and Classification (Kyu & Woraratpanya, 2020) has dataset with 1100 images or another work Comparison of Current Convolutional Neural Network Architectures for Classification of Damaged and Undamaged Cars (Ünal et al., 2022) used only 160 images and used training accuracy instead of validation one, what is a critical mistake and does not reflect the actual behavior of the model on unseen data. Also, many authors used the accuracy metric on the validation set as a measure of the quality of their model, which, due to the low diversity of the data, says very little about the most important thing - the ability of the model to generalize the acquired knowledge and use it on new unseen data. So I would use the third test dataset[5] to benchmark a model performance, it also allows me to compare different model architectures between each other, regardless on random split of training and validation sets. Furthermore, many papers from Graph 1 were used for inspiration.

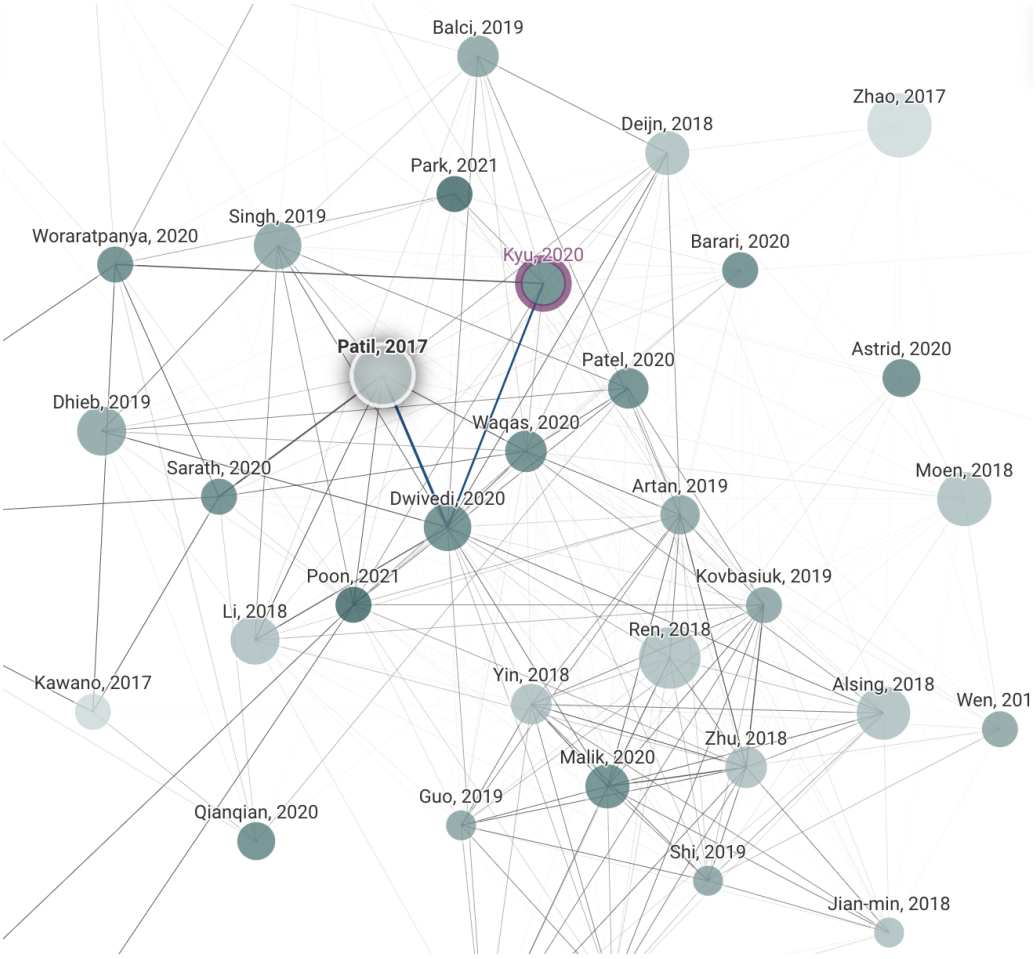


Figure 1: Related works for Deep Learning Based Car Damage Classification founded on connectedpapers.com (Author, 2023).

[5]The test set is used to approximate the models's true performance in the wild (Wilber, n.d.).

Also many solutions can be found on Kaggle that is most popular place for machine learning competitions with open-source code solutions. Baseline for binary classification task: damaged and undamaged cars starts from 80% accuracy on validation set and goes up to 90% on Kaggle. The same is true for scientific papers where the highest accuracy rate is 95.22% (Kyu & Woraratpanya, 2020) but unlike solutions from Kaggle, they can't be reproduced.

2 Data

2.1 Data description

The dataset used in this research contains photos of vehicles collected from various sources, including European ad services and American auctions. Dataset was assembled to develop an image recognition system distinguishing between clean and damaged cars.

The images in dataset were sourced from three primary channels: European ad platforms like Autoscout24 and MobileDe. It was the leading source for clean (undamaged) vehicles, while American auctions served as a rich source of damaged vehicle images. The last and the smallest source of mixed data was open-source datasets on Kaggle and Stanford Cars Dataset[6] (Krause et al., 2013).

One noteworthy characteristic of this dataset is the inherent class imbalance between clean and damaged vehicles. This imbalance arises from the nature of the data sources, as European ad platforms primarily showcase undamaged cars, while American auctions feature a higher proportion of damaged vehicles. Bellow in Figure 2 is shown a small representation sample:

Figure 2: Mixed sample of training data (Author, 2024)

2.2 Transformation and Normalization

In this work, I would use mainly the pre-trained weights for further model and the technique called transfer learning, when the body or a main part of a net was already trained on the ImageNet dataset with 14,197,122 annotated images and 1,000 classes according to the Stanford Vision Lab (VisionLab, 2009). ImageNet dataset images have 224x224 pixels resolution, and it's essential to transform pictures to similar ones because the network is expected to get the exact resolution and may have a poor performance due to different shapes. So, all images from my dataset must be normalized to 224x224 pixels and have a similar normalized colors to the ImageNet images. This process will be discussed in more details in section 5.2.

[6]The original link to dataset is broken.

2.3 Quality of data

The initial batch of data was provided by Carvago s.r.o. Initial training experiments began with a dataset containing approximately 6,000 images each for clean and damaged vehicles. Following an unsuccessful hyperparameter search, I hypothesized that data quality could impact model performance and proceeded with a manual review of the images. This review revealed that approximately 6.1% of vehicles in the clean folder were actually damaged to varying extents, and around 4.5% of vehicles in the damaged folder were undamaged. This mislabeling distribution is visualized in Figure 3.

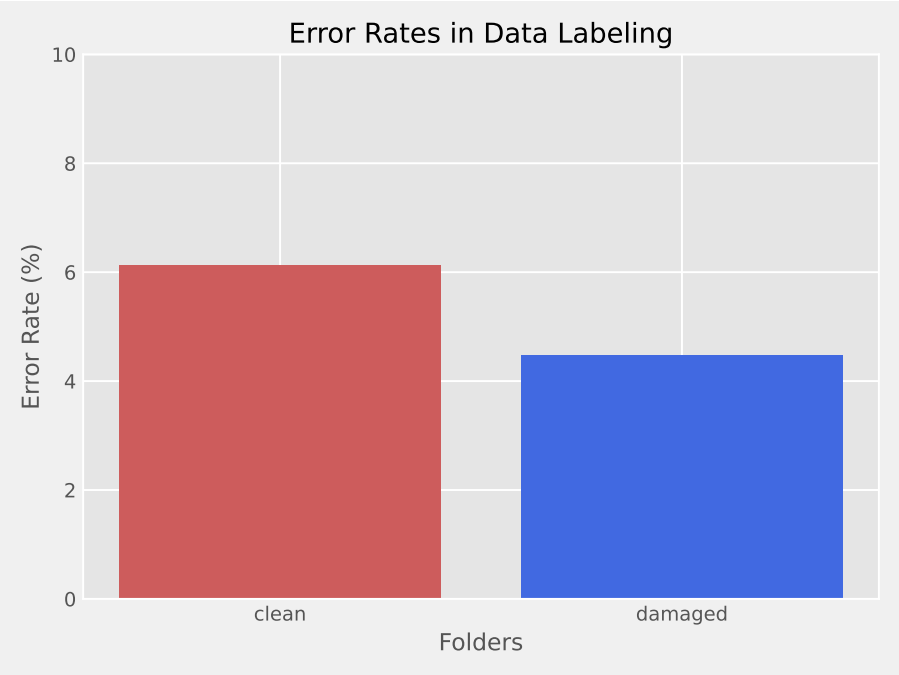


Figure 3: Percent of incorrect labels in Clean and Damaged folders (Author, 2023)

The problem with labeling is the most common one, even within popular open-source datasets. The issue was mentioned in the work (Curtis G. Northcutt, 2020). They identify label errors in the test sets of 10 of the most commonly used computer vision, natural language, and audio datasets, and subsequently study the potential for these label errors to affect benchmark results. Errors in test sets are numerous and widespread: we estimate an average of at least 3.3% errors across the 10 datasets, where, for example, label errors comprise at least 6% of the ImageNet validation set (Curtis G. Northcutt, 2020).

Also, they find that lower capacity models may be practically more useful than higher capacity models in real-world datasets with high proportions of erroneously labeled data. For example, on ImageNet with corrected labels: ResNet-18 outperforms ResNet-50 if the prevalence of originally mislabeled test examples increases by just 6% (Curtis G. Northcutt, 2020).

Ensuring high-quality training data is critical in deep learning. Looking ahead, not a single change in either the model or the hyperparameters led to the exact intense change in the accuracy of the net as revision of data labeling. This refinement increased the baseline accuracy of a stock ResNet-50 model from 82.63% to 86.91%.

2.4 Size of data

The final, revisioned training dataset was expanded with additional scraped images and samples from other open-source datasets, containing 44,600 images, or 22,300 images per class, all in JPG format. Every image was validated by humans in the appropriate flow shown described in Section 5.2. For the test dataset, a random sample of size 1,000 images was selected from the clean and damaged folders using Python's *random* module. These test images were removed from the training dataset to ensure no overlap between training and test sets.

3 Methodology - Introduction to Neural Networks

3.1 Evolution of Neural architectures in the vision domain

The field of computer vision has evolved and developed rapidly over the past two decades. Not only have the models themselves changed, but their underlying architectures as well, transitioning from CNNs to transformers in the early 2020s.

At one point, solving classification tasks was an insurmountable challenge for computers. Microsoft even created the Assira CAPTCHA based on this problem, where users had to identify cats or dogs as a form of verification. In 2008 state of the art classifier allows to solve a 12-image Asirra challenge automatically only with probability 10.3%. (Golle, 2008)

The advent of convolutional neural networks (CNNs) rendered the Assira CAPTCHA obsolete, and in 2014, Microsoft discontinued the project due to its ineffectiveness. Transformers have further enhanced the already powerful models. The Figure 4 shows the progression of computer vision models over time, highlighting key architectures and their release years. It divides the timeline into two eras: the CNN era (spanning from 1998 to 2019) and the Transformer era (from 2020 onwards). Notable models such as LeNet, AlexNet, ResNet, and ConvNeXt are labeled along with their respective release years, marking significant milestones in the evolution of neural networks for computer vision.

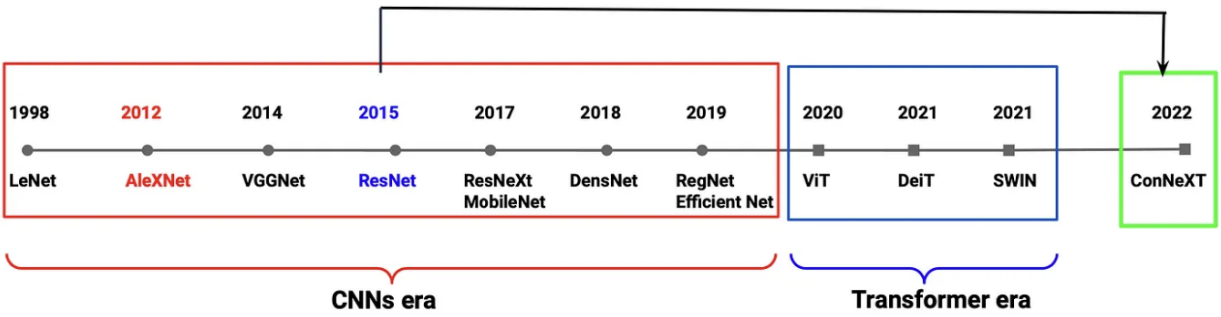


Figure 4: Evolution of Neural architectures in the vision domain (Ulyankin, 2023)

This paper will focus on CNN architectures for solving classification problems. Models selected for benchmarking are listed in Table 1. The reason why these models were selected is a difference in release date, highlighting the shift in baseline performance over time, and for their balance between accuracy and parameter efficiency. Table 1 includes key metrics such as Acc@1, Acc@5, and GFLOPS. Acc@1 refers to top-1 accuracy, which measures the percentage of cases where the model's top prediction matches the ground truth. Acc@5, on the other hand, refers to top-5 accuracy, where the ground truth is present within the model's top 5 predictions. GFLOPS, or giga floating-point operations per second, indicates the computational cost of the model, serving as a measure of efficiency.

Table 1: Table of classification weights (PyTorch Vision Team, n.d.-c)

Weight	Acc@1	Acc@5	Params	GFLOPS
`ResNet50_Weights.IMAGENET1K_V1`	77.618	93.698	25.0M	4.23
`ConvNeXt_Tiny_Weights.IMAGENET1K_V1`	82.52	96.146	28.6M	4.46

3.2 Neural Networks

3.2.1 Neuron model and typical activation functions

A neural network is a giant abstraction for describing a complex function, where at each step, certain transformations are applied to the data, for example, a linear regression function followed by the application of a nonlinear activation function (sigmoid or ReLU[7]). Like most functions, a specific value x_n, where n is an index of the feature vector, is taken as input. After some calculations, the output is y. In other words, a neural network aims to approximate a function f able to map an input feature vector x to an output class $\hat{y}$.

- x_n - input *or feature in our case*
- w_n - weight that changing during a training
- $\hat{\beta}$ - fixed bias that stayed unchanged during a training
- y - output value of function or confidence
- $\hat{y}$ - output value after non-linear activation function applied, shows actual prediction

Calculation of y value for the one specific neuron, where $\hat{\beta}$ is a fixed bias:

$$y = x_1 \cdot w_1 + x_2 \cdot w_2 \dots x_n \cdot w_n + \hat{\beta} \tag{1}$$

Calculate output $\hat{y}$ by applying a sigmoid activation function:

$$\hat{y} = \sigma(y) = \sigma(x_1 \cdot w_1 + x_2 \cdot w_2 \dots x_n \cdot w_n + \hat{\beta}) \tag{2}$$

Historically, a common choice of activation function is the Sigmoid function σ. The sigmoid function converts the values it receives to the real range [0, 1]. If the input data turns out to be large positive values, then after conversion they will be equal to approximately one, and negative numbers will be close to zero.

[7]ReLU - Rectified Linear Unit

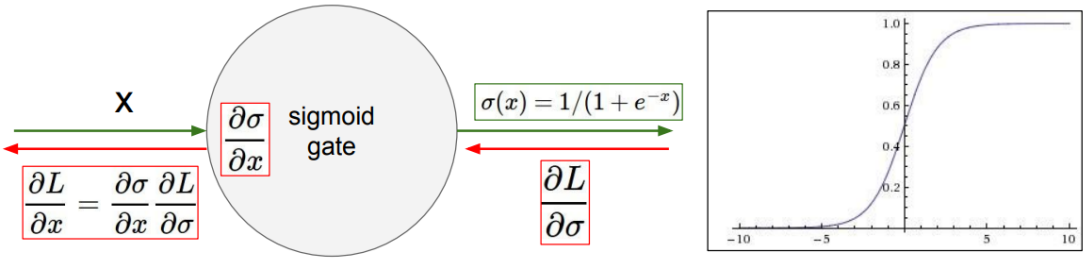


Figure 5: Forward and Back propagation with Sigmoid activation function (Stanford University, 2017)

The sigmoid non-linearity has declined in popularity and is now rarely used in neural networks. Instead, functions like ReLU are much more common. This topic will be discussed in more detail in Section 4.1, which covers activation functions.

3.2.2 Fully connected Neural Networks

A neural network is composed of a collection of neurons that are connected in an acyclic graph and organized in layers. According to this structure, the outputs of some neurons can become inputs to other neurons, as shown in Figure 6 where the scheme of example neural net topology is reported. The architecture of a common FCNN[8] can be decomposed in three basic elements:

- an input layer
- one or more hidden layers
- an output layer

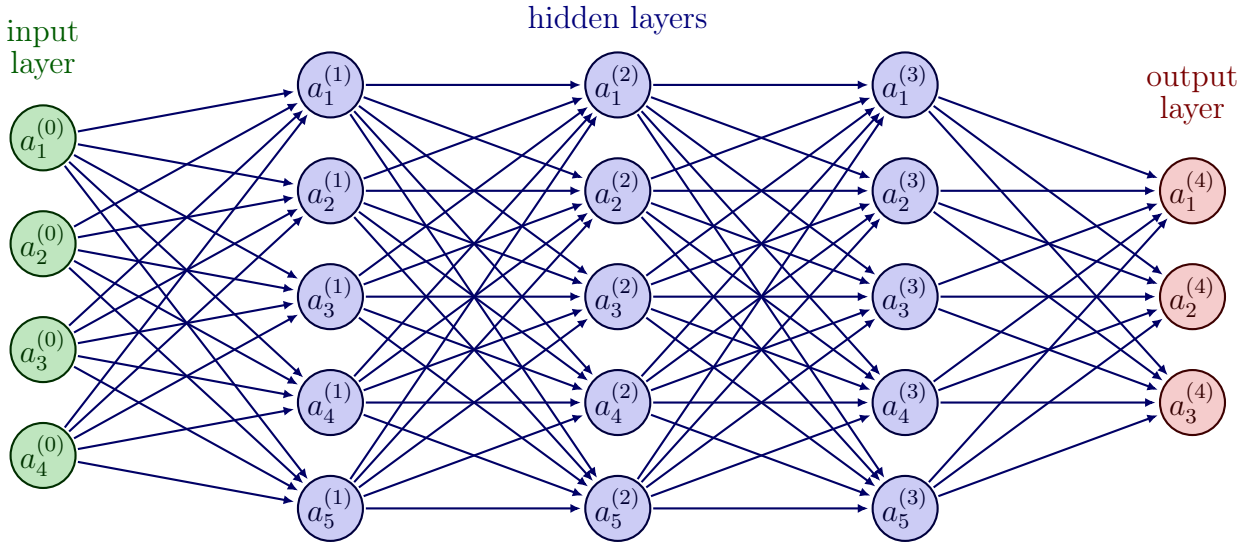


Figure 6: Common fully connected neural network with 3 hidden layers (Author, 2023)

[8]FCNN - fully connected neural network

As already was said, an input layer is waiting for X_n features in some standard format, for example normilized vectors of image in range $[-1; 1]$ to make further computing much easier. The hidden layers are responsible for identifying patterns from input data. The count of hidden neurons also determines how the complex pattern network can identify from the input data, which is shown in Figure 7. However, too many neurons may cause overfitting and the model's inability to generalize acquired knowledge. The phenomenon of overfitting will be discussed in more detail in the 4.5 chapter. An output layer serves for prediction or mapping the data obtained from previous layers to a specific class. That is why the output layer neurons most commonly do not have an activation function like ReLU. Hence, with an appropriate loss function that quantifies the agreement between the predicted scores and the ground truth labels on the neuron's output, we can turn a single neuron into a linear classifier. In other words, we cast this as an optimization problem in which we will minimize the loss function with respect to the parameters of the score function.

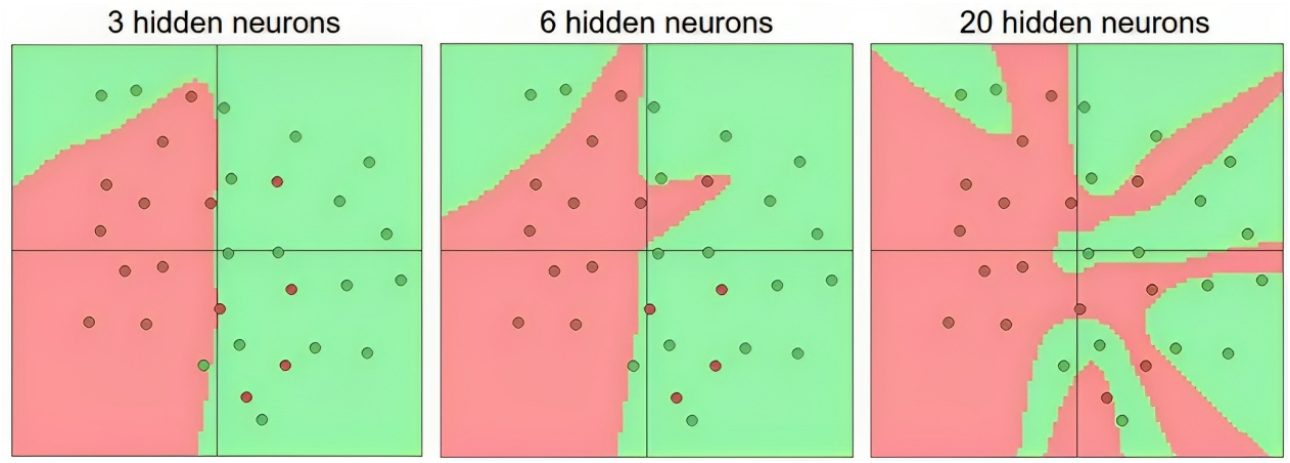


Figure 7: Larger Neural Networks can represent more complicated functions. The data are shown as circles colored by their class, and the decision regions by a trained neural network are shown underneath (Stanford University, 2017)

3.3 Model architecture CNN

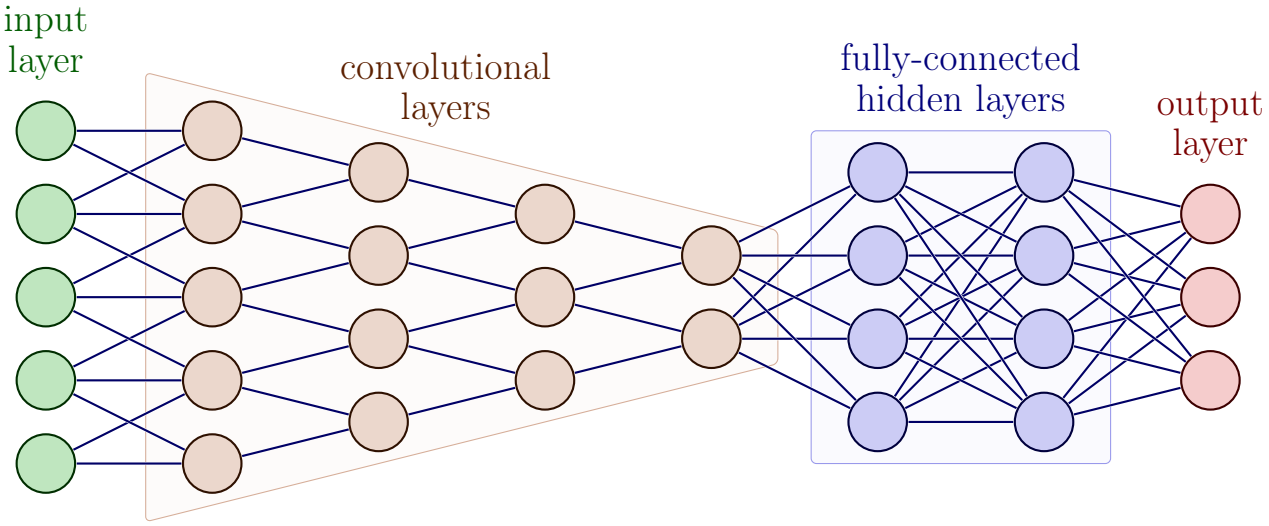


Figure 8: Common convolutional neural network (Author, 2023)

Before the advent of Convolutional Neural Networks, one of the major challenges in computer vision was the inability of models to handle the spatial variability of objects within images. This issue arose from the way images are represented in computers. An image is essentially a matrix of pixel values ranging from 0 to 255, which represent brightness levels (in practice, these values are often normalized to improve computational efficiency). Figure 9 illustrates this concept using a grayscale image.

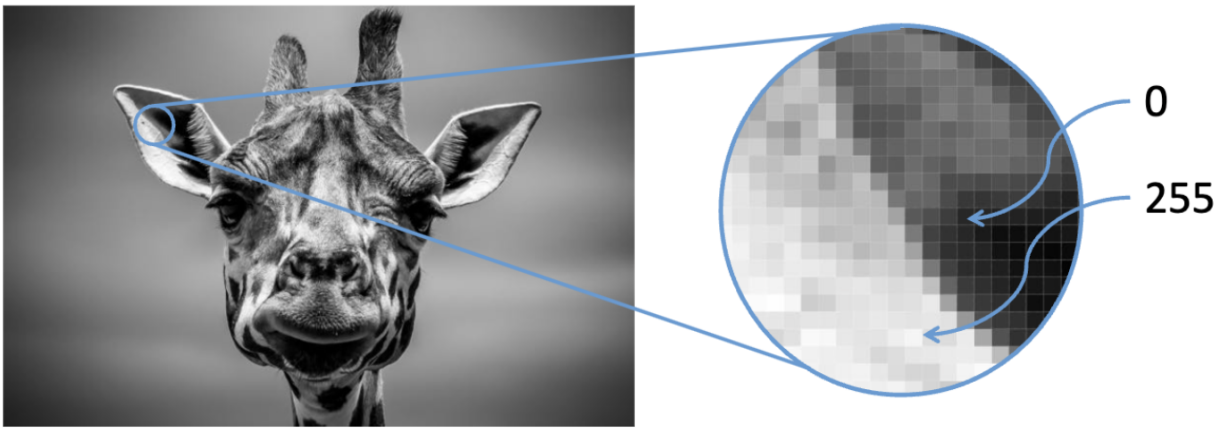


Figure 9: Image is a matrix (Andrey Zimovnov, 2018).

When dealing with colored images, an additional dimension is introduced. Since colors are a combination of red, green, and blue channels — known as RGB — the image becomes a 3D tensor. Figure 10 illustrates this new dimension in a matrix format. Additionally, the PNG[9] format supports a fourth channel, known as the alpha channel, which represents transparency. This added dimension can cause issues during transformation processes, as many image processing techniques are designed to work with three channels (RGB). If the alpha channel is not properly handled or removed, it can lead to unexpected behaviors or incorrect feature extraction.

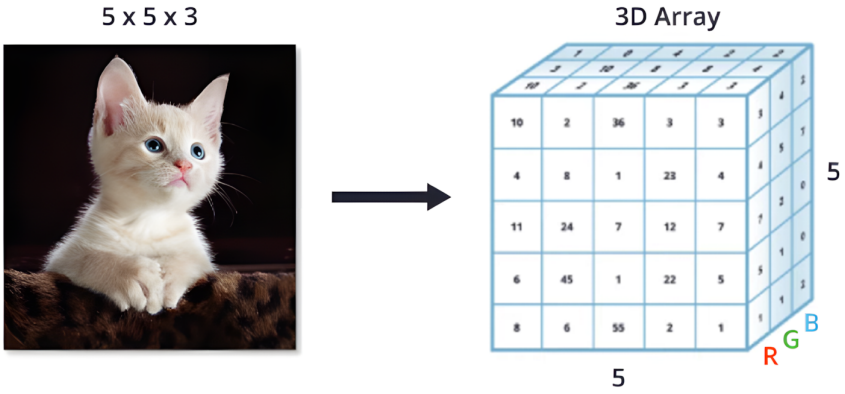


Figure 10: RGB Image is a tensor (Andrey Zimovnov, 2018).

A significant challenge arises during the feature extraction phase. A traditional approach was to flatten the image matrix into a 1D vector (also known as a feature vector). For instance, a 128x128 pixel image would generate 16,384 features in the form of vector:

$$\begin{bmatrix} X & Y & Z & \dots & 0 & 0 \end{bmatrix}$$

This vector would then be processed through fully connected (FC) layers with activation functions to produce the output.

[9]PNG (Portable Network Graphics) is a raster-graphics file format that supports lossless data compression.

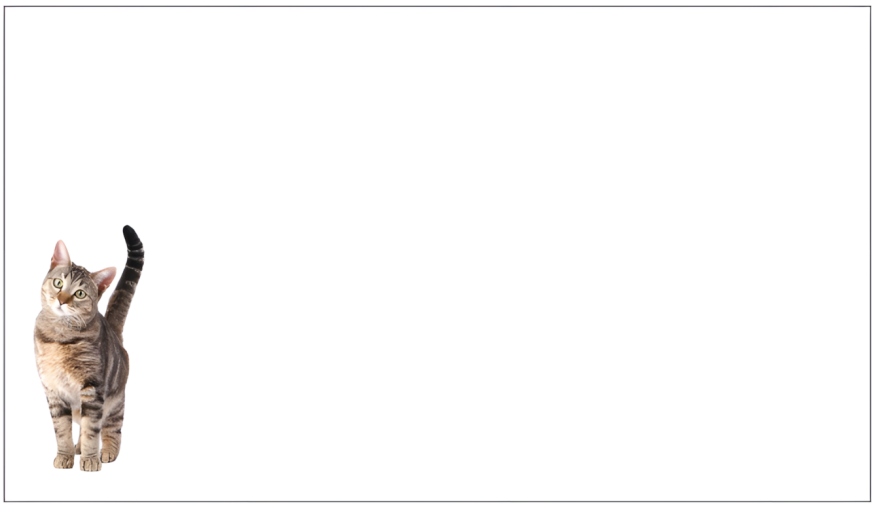


Figure 11: Basic image (Andrey Zimovnov, 2018).

However, this approach has several critical disadvantages. If the object in the image is shifted, as shown in Figure 11 and Figure 12, the feature vector changes completely, making it difficult for the model to generalize — a key property of neural networks. Additionally, this method introduces other issues, such as an excessive number of parameters, which increases training time and model complexity, risks of overfitting, and the loss of spatial relationships between neighboring pixels.

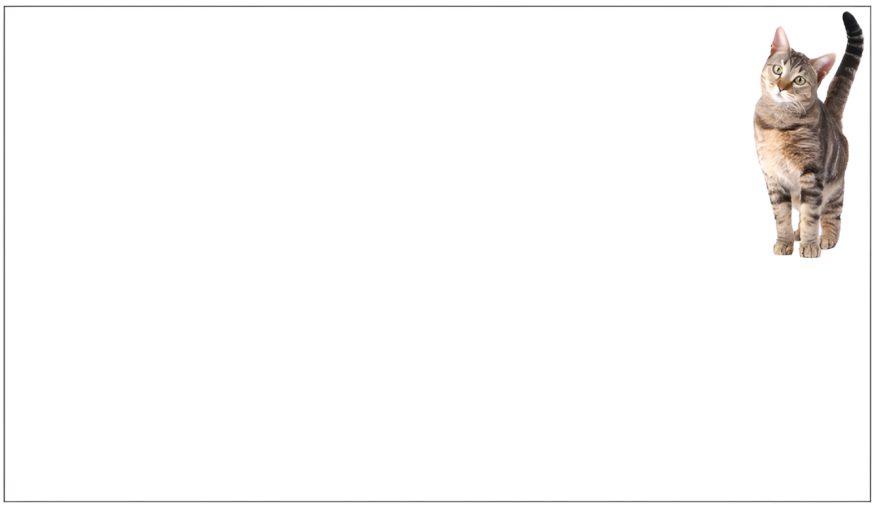


Figure 12: Shifted image (Andrey Zimovnov, 2018).

The architecture of CNN allows extracting complex and abstract patterns from image data relatively quickly respect to other conventional methods. Convolutional Neural Networks, in fact, have become one of the most used tool in computer vision field for solving problems of visual recognition, such as image classification, object detection and recognition. CNNs do pattern extraction and handle image shift with convolutional layers block, yellow part on Figure 8.

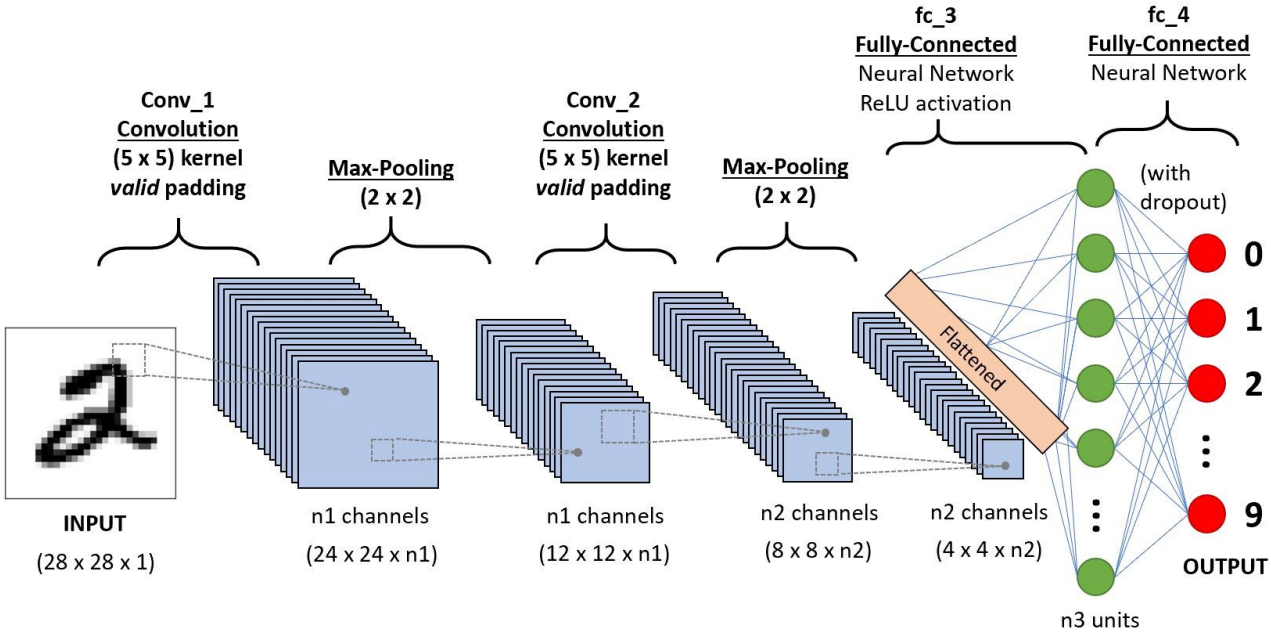


Figure 13: Architecture of the Convolutional Neural Network (Ratan, 2020)

CNNs exploit the convolutional operator instead of the general matrix multiplication in at least one of the layers. Their structure (Figure 13) can be divided into two main blocks:

- the first block, made up of a sequence of convolutional layers inter-leaved with non-linear functions and some sub-sampling layers (respectively ReLU and Max Pooling in Figure 13), is aimed at learning patterns and local and spatial features directly from the annotated training images that are input to the network
- the second block, used to actually implement the visual recognition task, is a series of fully connected layers which makes the aggregation of the previous convolutional map output and get definitive scores output for a class, through the classification probability distribution

3.3.1 Convolutional layers

In the first section of a CNN, the convolutional layers are organized in a sort of hierarchical structure, meaning that the first layers are dedicated to learning low-level features, such as edges, color, gradient orientation, curves, while, going up with the layers, the learnt features become more and more complex, like corners (mid-level), blobs (high-level). The role of the CNN is to reduce the images into a form which is easier to process, without losing features which are critical for getting a good prediction. In fact, thanks to the introduction of the convolutional part, CNNs can learn even complex features of a set of images, that are seen as compositions of simpler features, directly from the data, without requiring the effort of extracting them manually. (Vaiana, 2020)

The element implicated in performing the convolution operation in the first part of a Convolutional Layer is called the *Kernel/Filter*, as illustrated in Figure 14.

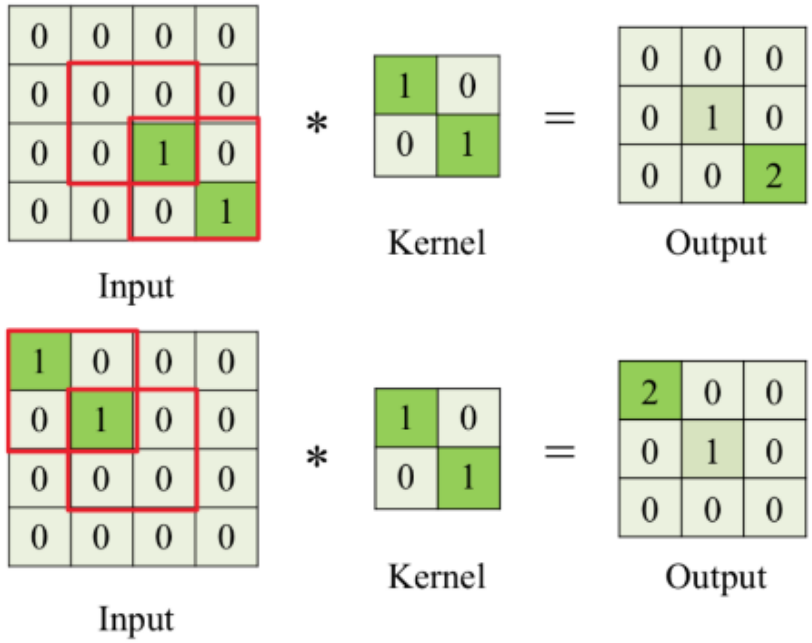


Figure 14: Filtering operation in Convolutional layer and position irrelevance (Stanford University, 2017)

Every filter is small spatially (along width and height), it is applied and slided on the entire extension of the input image in order to learn a specific feature. Therefore, the convolutional layer's parameters consist of a set of learnable filters. The filtering operation is performed through a series of multiplications and sums of the input values by some parameters, called weights, whose values identify the features to be learnt. For example, a typical filter on a first layer of a CNN might have size 5x5x3 (i.e. 5 pixels width and height, and 3 because RGB images have depth 3, the color channels).

During the forward pass, we convolve each filter across the width and height of the input volume and compute dot products between the entries of the filter and the input at any position: the convolution operation is applied. Then, each intermediate result, given by all the multiplications, is summed in order to provide the final result. After its storage, it is located into an output matrix: *feature map* or activation map. During the forward pass, we convolve each filter across the width and height of the input volume and compute dot products between the entries of the filter and the input at any position. As we slide the filter over the width and height of the input volume we will produce a 2-dimensional activation map that gives the responses of that filter at every spatial position. Actually, *three hyperparameters* control the size of the output volume: the depth, the stride and zero-padding. (Vaiana, 2020)

1. the *depth* of the output volume corresponds to the number of filters we would like to use, each learning to look for something different in the input. For example, if the first Convolutional Layer takes as input the raw image, then different neurons along the depth dimension may activate in presence of various oriented edges, or blobs of color. A set

of neurons that are all looking at the same region of the input is called depth column. Parameter n1 and n2 on Figure 13.

2. the stride is referred to how the filter is going through the input layer. When the stride is 1 then we move the filters one pixel at a time, while if the stride is 2 then the filters jump 2 pixels at a time, so that they produce smaller output volumes spatially

3. the feature of zero-padding is that it will allow us to control the spatial size of the output volumes, by padding the input volume with zeros around the border; the main reason of this possible choise is that it is useful to preserve the spatial size of the input volume, that so the input and output width and height are the same

The spatial size of the output volume can be calculated through the formula:

$$\left[\frac{N + 2P - F}{S}\right] + 1$$

as a function of the input volume size, in terms of width/height (N), the receptive field size of the Convolutional Layer neurons (F), the stride with which they are applied (S) and the amount of zero padding used (P) on the border ("COMPSCI 682 Neural Networks: A Modern Introduction", 2020).

To detect larger objects in an image, the receptive field must be expanded by increasing the number of convolutional layers or kernel window size, Figure 15 demonstrates this concept. Receptive Field (RF) is defined as the size of the region in the input that produces the feature. Basically, it is a measure of association of an output feature (of any layer) to the input region (patch) (Adaloglou, 2020). In the context of classifying intact and damaged cars, an appropriate receptive field is critical. It must be large enough to capture the entire structure of a car for context, while also identifying local features like cracks or dents that signify damage.

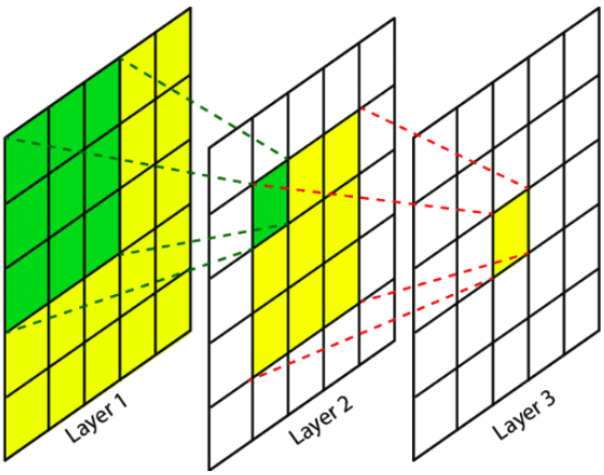


Figure 15: Receptive field with 3 convolution layers (Adaloglou, 2020)

This is a crucial aspect, as some objects may not even be detected by the model, leading to errors and poor performance. As Araujo et al. (2019) aptly describes: *"We observe a logarithmic*

relationship between classification accuracy and receptive field size, suggesting that while large receptive fields are necessary for high-level recognition tasks, the returns diminish as the field grows larger". Additionally, Figure 16 illustrates this relationship across several different models in the computer vision domain, where size of the bubble represents a model complexity or a number of parameters.

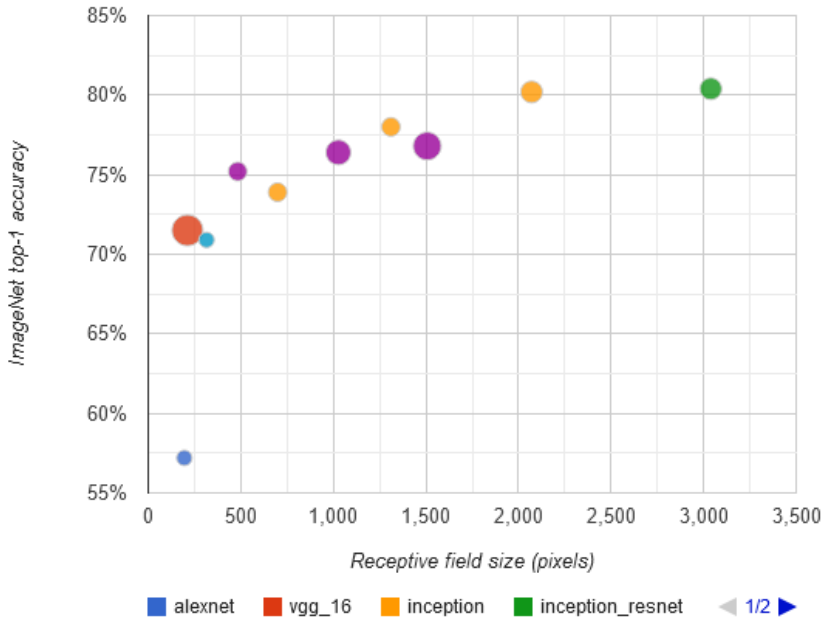


Figure 16: Relationship between classification acc@1 and receptive field size (Adaloglou, 2020)

An increase in the receptive field size often comes with drawbacks, such as a significant increase in the number of parameters within the neural network. To address this issue, several aggressive strategies have been proposed. One such approach is the use of a *stride*, which, as previously mentioned, allows the model to skip certain pixels. This approach is based on the assumption that neighboring pixels may not contribute substantially to the final output. A more effective alternative, however, is the application of pooling techniques.

3.3.2 Pooling

The idea of pooling is to get the most valuable and relevant patterns from images with the lowest computing cost. When the kernel window slides on the image, the pooling layer calculates the max or average for the window, creating a new features map with the most important patterns and rapidly decreasing image size, simultaneously increasing the receptive field. Figure 17 demonstrates image dimensionality drop after applying pooling layer. Common CNN displayed in Figure 13 has two *Max-Pooling* layers, thanks to which the dimension drops four times.

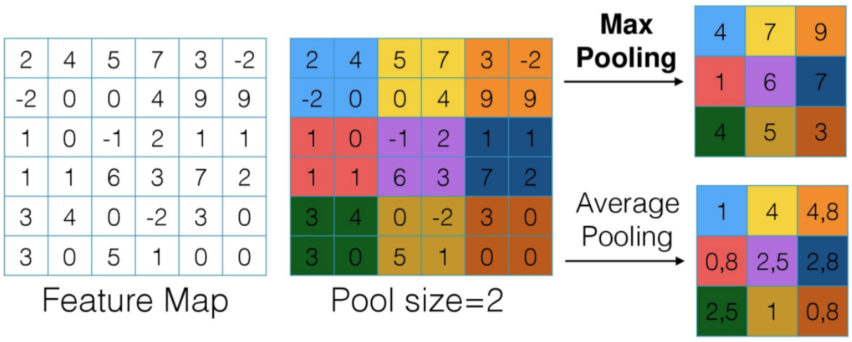


Figure 17: Pooling layer (2x2). MAX and AVG methods (Stanford University, 2017)

3.3.3 Summary

Advent of Convolutional Neural Network resolve many issues in Computer Vision domain such as:

- object shift tolerance with convolutional kernels that can extract common patterns from images and pooling layers which reduce the spatial dimensions of the feature map by summarizing the most important features in local regions
- parameter efficiency: unlike fully connected networks, CNNs significantly reduce the number of parameters through weight sharing. Filters are reused across different spatial regions, allowing CNNs to learn with fewer parameters

4 Methodology - setting up a neural network

4.1 Activation Functions

The choice of activation function could be very important in neural network training. The sigmoid function, though once widely used, has limitations: its derivative peaks at 0.25 as shown in Equation 3, often causing vanishing gradients in deeper networks and limiting weight updates. Additionally, the sigmoid function is more computationally complex, than ReLU like (Weights & Biases, 2020).

$$\frac{\delta\sigma}{\delta x} = \sigma(x) \cdot (1 - \sigma(x)) \leq 0.25 \tag{3}$$

These issues can be resolved by using ReLU (see Figure 18), which is defined by the formula:

$$\text{ReLU}(x) = \max(0, x) \tag{4}$$

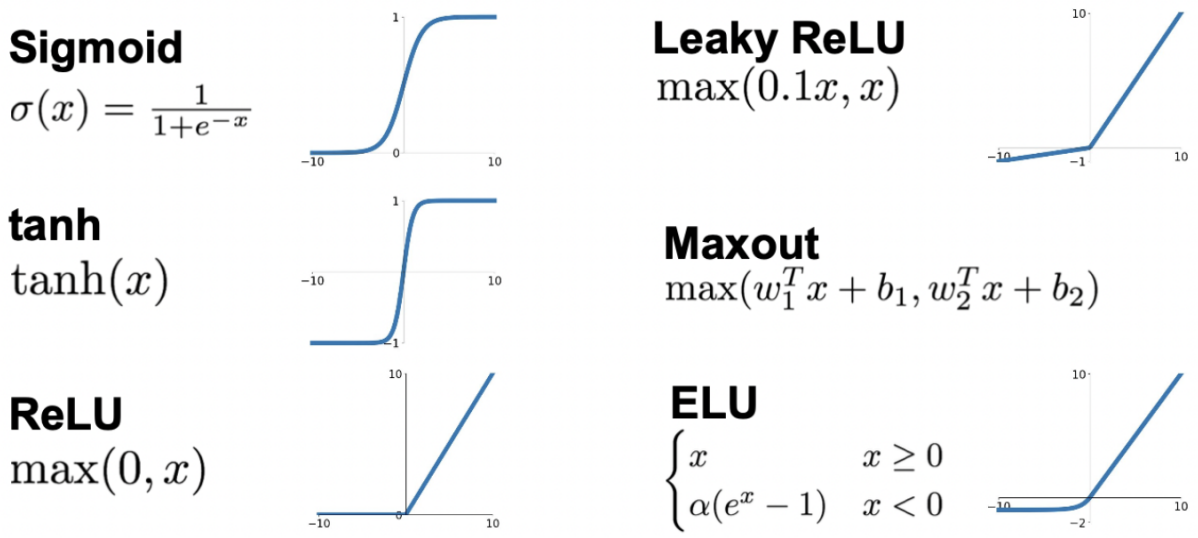


Figure 18: Activation functions (Stanford University, 2017)

In this paper I will use pre-trained relatively new neural networks, one of the state-of-the-art activation function is already there. ConvNeXt uses the Gaussian Error Linear Unit (GELU) activation function. GELU is a non-linear activation function that is smoother and more gradual than the ReLU activation function. Figure 19 is illustrating it.

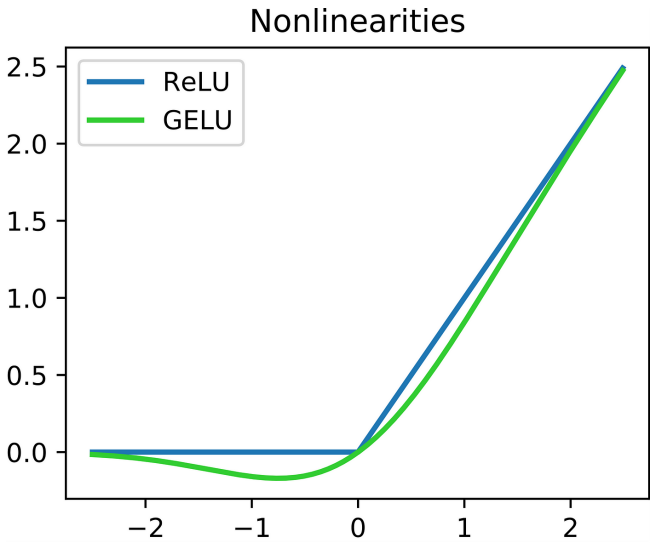


Figure 19: Comparison of ReLU and GELU activation functions

Tests show that on numerous basic dataset like CIFAR10, CIFAR100 and ImageNet1k the GELU exceeded the accuracy of the ELU and ReLU consistently, making it a viable alternative to previous nonlinearities (Hendrycks & Gimpel, 2016). GELU will be employed in practical part as an activation function for ConvNeXt classifier.

4.2 Learning Rate

The learning rate, commonly denoted as η, defines the step size an optimizer takes while minimizing a loss function using gradient descent or its advanced variations. It directly influences the convergence speed and stability of the training process. The graphs in Figure 20 illustrate how different fixed learning rate values affect the behavior of the loss function. Details on the optimization process are discussed in Section 4.3.

There are two primary methods for determining an optimal learning rate. The first is a *monotonic grid-search*, which involves gradually decreasing the learning rate until the model can no longer escape from a local minimum. While effective, this method requires considerable time and computational resources. The second approach uses a *learning rate finder* based on cyclic learning rate principles.

- too high learning rate leads to divergence of loss value
- optimal learning rate shows balance between decreasing a loss value and numbers of epochs to achive it
- too low learning rat can produce stable learning but may cause the model to get trapped in local minima, resulting in early training stagnation

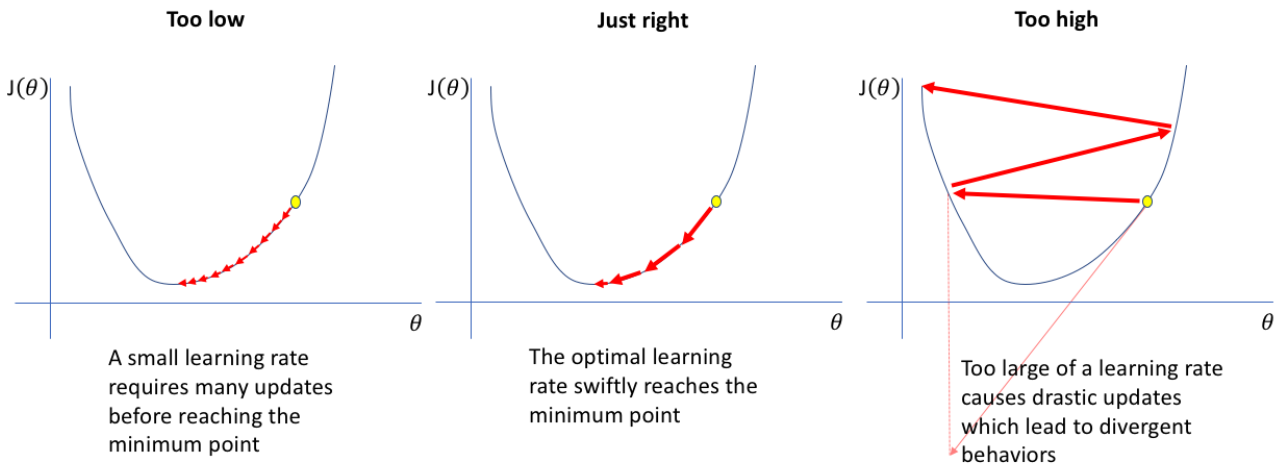


Figure 20: Learning rate value impact on the value of loss function (Jermey Jordan, 2018).

A more efficient approach is the cyclic learning rate (CLR) from (Smith, 2017). Which not only saves a lot of time and computational resources by making a good estimation for learning rate value, but also speeds up learning by several times, as in the example of the CIFAR-10 dataset (Smith, 2017). Figure 22 shows how CLR outperform static and exponential learning rates.

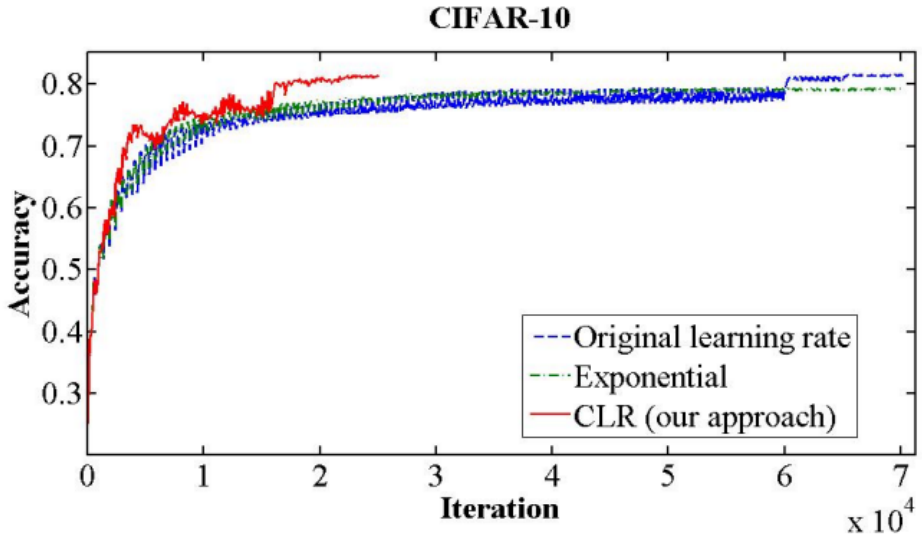


Figure 21: Classification accuracy while training CIFAR-10. The red curve shows the result of training with one of the new learning rate policies (Smith, 2017).

The essence of this learning rate policy comes from the the observation that increasing the learning rate might have a short-term negative effect and yet achieve a longer-term beneficial effect. This observation leads to the idea of letting the learning rate vary within a range of values rather than adopting a stepwise fixed or exponentially decreasing value. That is, one sets minimum and maximum boundaries, and the learning rate cyclically varies between these bounds. (Smith, 2017)

CLR (cyclic learning rate) helps escape a local minimum and plateaus by changing the

learning rate value between two boundaries (initial_lr and min_lr) with a linear or exponential scale. The behaviour of learning rate in this scenario is defined by a *scheduler* object with an appropriate function. The figure 22 below shows the idea of CLR.

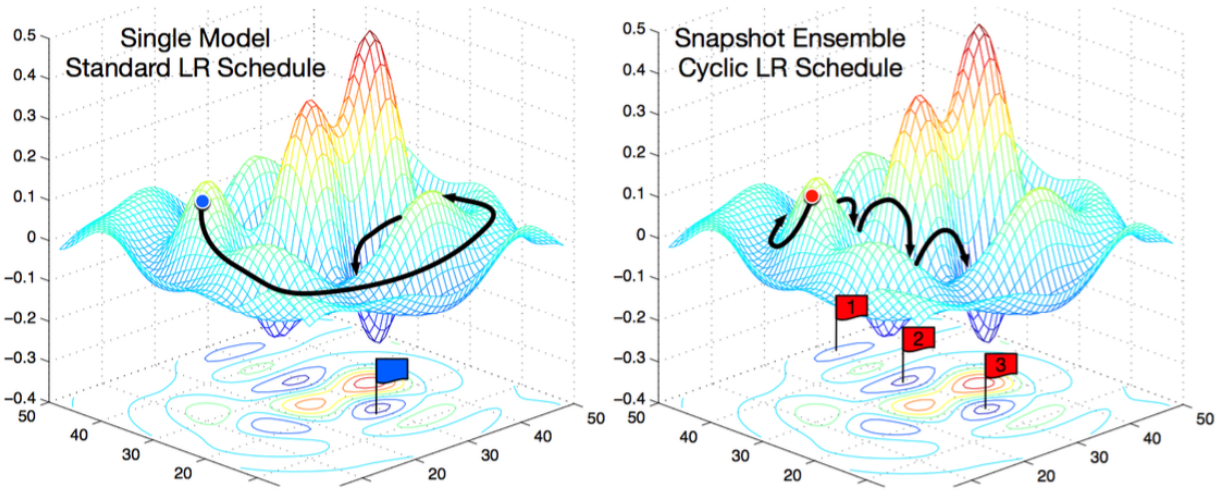


Figure 22: Escaping local minimum by increasing a learning rate value (Stanford University, 2017)

4.2.1 Scheduler

The *scheduler* is a function that takes an optimizer and defines the rate of change of the learning rate during training. The most common schedulers are shown below in the Figure 23.

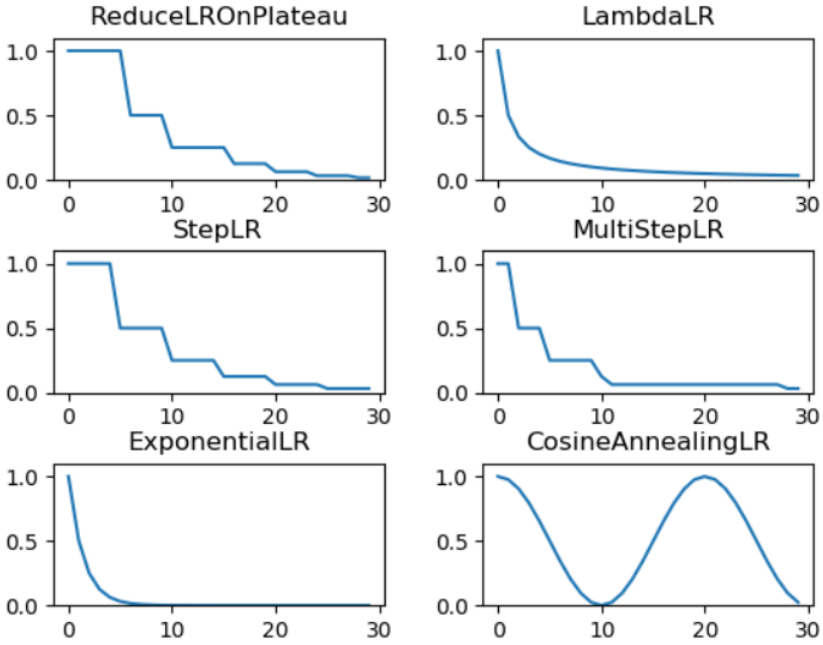


Figure 23: Learning rate schedulers (Stanford University, 2017)

Another common challenge in optimizing neural networks is the presence of *saddle points* — locations in the loss surface where the gradient is zero across all axes, but the point is neither a local minimum nor a maximum. These points often trap gradient-based optimization methods, causing slow convergence or even stagnation. Figure 24 represents this issue.

Schedulers can mitigate this issue by dynamically adjusting the learning rate over time. A larger learning rate at the early stages of training allows the optimizer to make significant progress across flatter regions, such as saddle points, without being trapped. As the learning rate decreases, the model can perform more fine-tuned adjustments near local minima. Also, *CosineAnnealingLR* (see Figure 23) like schedulers allow us to build an ensemble of models by finding more than one local minima. The *CosineAnnealingLR* scheduler will be used in practical part to improve overall performance of training.

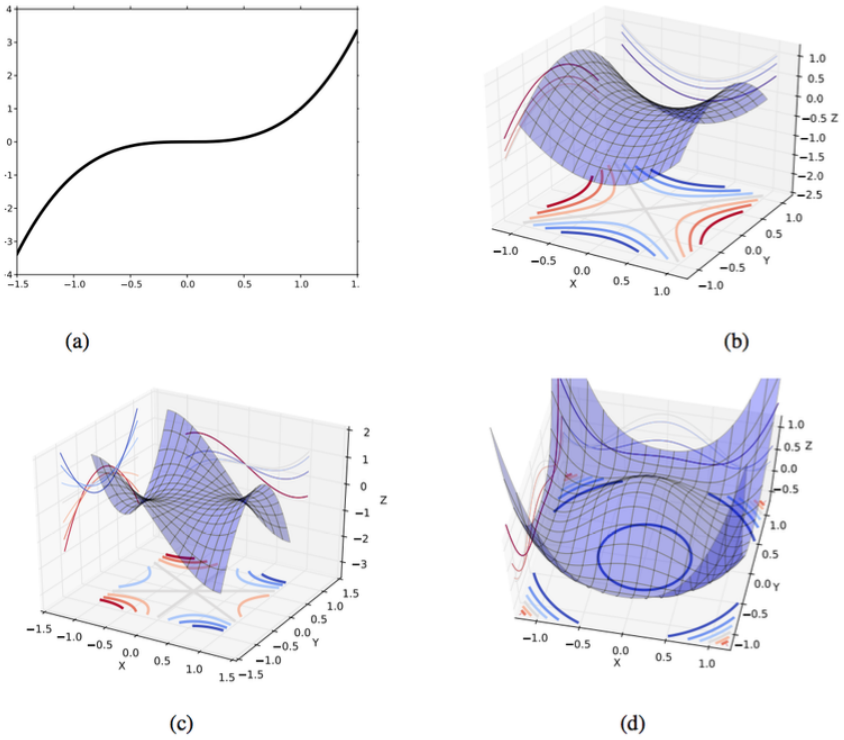


Figure 24: Example of the saddle point (Stanford University, 2017)

4.2.2 LR Finder

In essence, objective is to achieve a learning rate that results in a steep decrease in the network's loss. This can be discerned through an experiment where we gradually increase the learning rate after each mini-batch, recording the loss at each increment. This gradual increase can be on either a linear or exponential scale.

For learning rates that are too low, the loss may decrease, but at a very shallow rate. When entering the optimal learning rate zone, you'll observe a quick drop in the loss function. Increasing the learning rate further will cause an increase in the loss as the parameter updates cause the loss to "bounce around" and even diverge from the minima. The optimal learning

rate is associated with the steepest drop in loss - the middle of Figure 25, so we're mainly interested in analyzing the slope of the plot (Jermey Jordan, 2018).

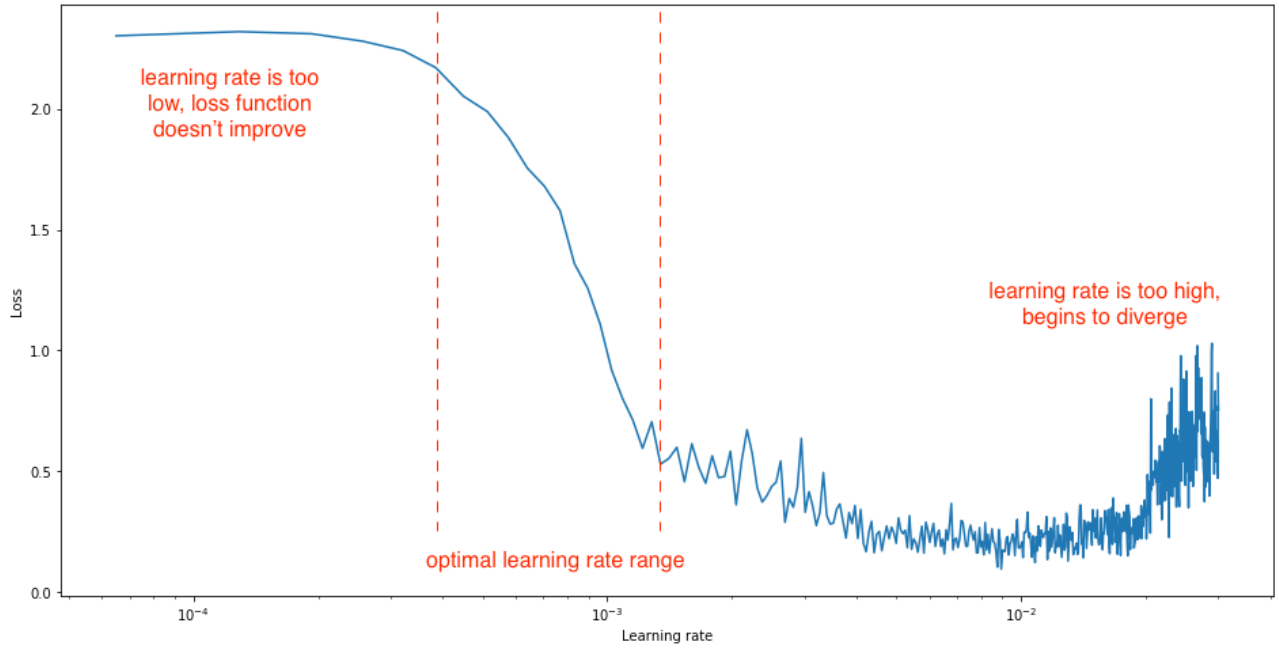


Figure 25: Optimal learning rate range (Jermey Jordan, 2018).

Implementation of the learning rate finder can be found in a python library *pytorch-lr-finder* (David Silva, Nale Raphael, 2020). This technique would be used in practical part to restrict the area of optimal learning rates and choose the initial one. With this way there is no need to perform an inefficient grid search.

4.3 Optimizer

4.3.1 Gradient Decent (GD) and loss function optimization

The optimization problem in machine learning is typically framed as finding the set of parameters w that minimize a given loss function $L(w)$. The loss function measures how far off the model's predictions are from the true values. Mathematically, it is expressed as:

$$L(w) = \sum_{i=1}^{n} L(w, x_i, y_i) \rightarrow \min_{w} \tag{5}$$

In order to minimize $L(w)$, we need to find the values of w that produce the lowest possible output of the loss function. The main tool used for this purpose is the gradient.

The gradient of a function is a vector that points in the direction of the steepest ascent, i.e., the direction in which the function increases the fastest. In the case of the loss function $L(w)$, the gradient $\nabla L(w)$ provides information about how to adjust the parameters w to increase the loss function.

For a given loss function *L(w)*, the gradient is calculated as:

$$\nabla L(w) = \left(\frac{\partial L(w)}{\partial w_0}, \frac{\partial L(w)}{\partial w_2}, \ldots, \frac{\partial L(w)}{\partial w_k} \right) \tag{6}$$

To minimize the loss function, we update the parameters w by moving in the opposite direction of the gradient. This iterative process is known as gradient descent. The update rule is:

$$w_t = w_{t-1} - \eta \cdot \nabla L(w_{t-1}) \tag{7}$$

Here are decsribed variables:

- w_t - represents the updated parameters at step t
- η - the learning rate, a small constant that controls the step size
- $\nabla L(w_{t-1})$ - the gradient of the loss function with respect to the parameters at the previous step $t-1$

By repeatedly applying this update rule, the model's parameters are adjusted, and the loss function value decreases until it reaches a minimum (or a point close to the minimum).

4.3.2 Stochastic gradient descent and Mini-batch SGD

Calculating the gradient of the loss function and updating the parameters at each gradient descent iteration can be computationally expensive - $O(n)$ for each sum, especially for large datasets or complex models. To reduce computation time, another techniques can be used such as Stochastic Gradient Descent (SGD).

SGD is an optimization technique that updates the model parameters based on the gradient computed from a randomly selected subset of the training data, known as a mini-batch (also a hyper-parameter, will be discussed in the next sections). Instead of calculating the gradient over the entire dataset (as in traditional gradient descent), SGD approximates the gradient using only a one observation or a small batch of data, which significantly reduces the computational cost per iteration.

By performing updates more frequently (at every mini-batch), SGD can converge faster, although with more noise in the gradient updates. The Figure 26 shows difference between GD and SGD, noise can be decreased with lower η - learning rate (LR). Nevertheless, sometimes noise can help escape local optimum and continue calculation.

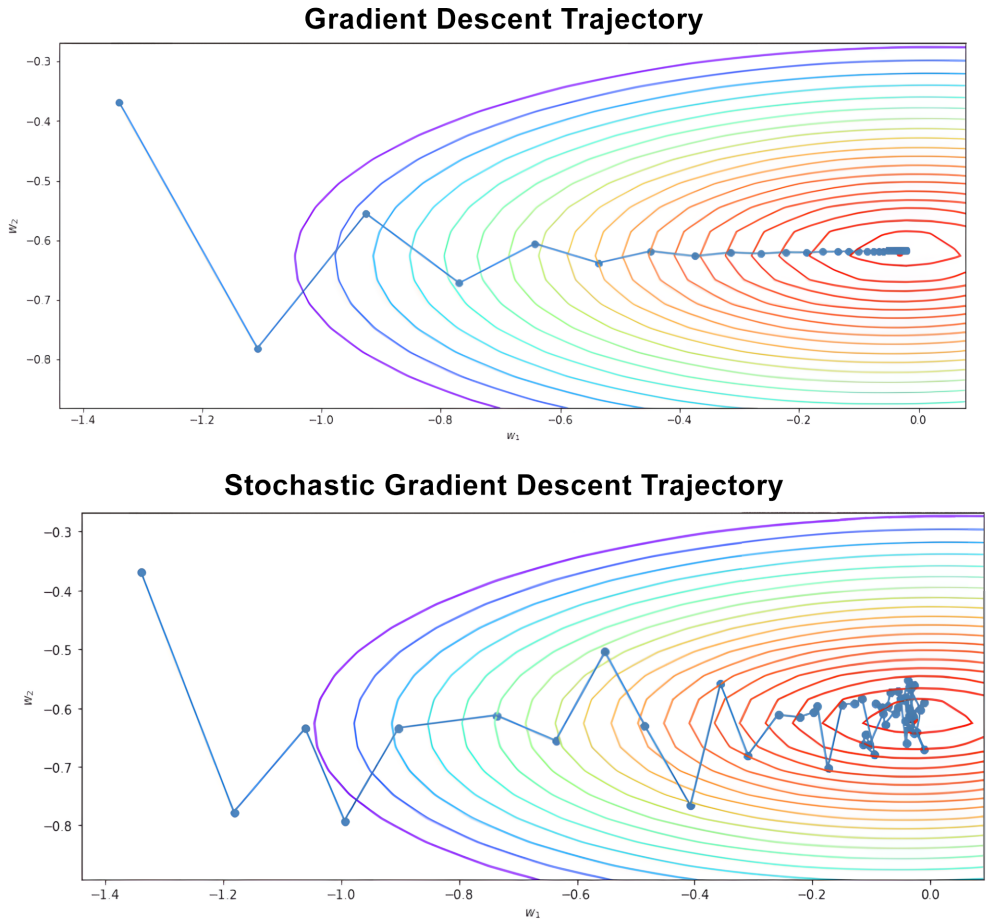


Figure 26: Trajectories of Gradient Descent (GD) and Stochastic Gradient Descent (SGD)

4.3.3 Momentum and Adaptive learning rate

Before take a look on the optimization baseline for today's machine learning, it's also worth mentioning the algorithms it was created on.

Momentum SGD allows the algorithm to move faster over curved surfaces by memorizing directions of previous gradients, avoiding the slowdowns caused by issues such as "pathological curvature" (e.g., narrow valleys, steep slopes, and local minima). As a result, iterations with momentum (α) can reach the optimum faster and smoother than with vanilla gradient descent (Goh, 2017). Here, the value of $\alpha \cdot h_{t-1}$ defines how strongly the previous gradient calculation should affect the present one and g_t is the gradient of the loss function at the current weights w_t:

$$h_t = \alpha \cdot h_{t-1} + \eta \cdot g_t \tag{8}$$

Then, make an optimization step with the momentum-augmented step h_t:

$$w_t = w_{t-1} - h_t \tag{9}$$

AdaGrad the algorithm that adaptivelly adjust learning rate on the variability of the parameters. In other words: the update step should be smaller for those parameters that vary to a greater extent in the data due to high uncertianty, and larger for those that are less variable.

The algorithm achieves this by accumulating the squares of gradients from previous steps. This is shown in equation bellow, where j refers to a specific parameter (or weight), which controls its individual learning rate and t refers to the iteration during training:

$$G_t^j = G_{t-1}^j + g_{tj}^2 \tag{10}$$

The optimization step for AdaGrad is shown bellow and involves adding a small constant ϵ to prevent division by zero:

$$w_t^j = w_{t-1}^j - \frac{\eta}{\sqrt{G_t^j + \epsilon}} \cdot g_t^j \tag{11}$$

However, this leads to a drawback: as the sum of the squared gradients grows, it causes the learning rate to decrease over time, eventually resulting in vanishing gradients, where each subsequent update becomes smaller and approaches zero.

RMSprop[10] and ADAM algorithms fix this issue by adding an exponential smoothing to the optimization process.

4.3.4 Adaptive momentum estimation (ADAM)

ADAM combines momentum feature from Momentum SGD and individaul learning rate from AdaGrad or RMSprop with exponential smoothing. Exponential smoothing is a technique used to weigh recent data points more heavily than older ones, providing a balance between responsiveness to new gradients and stability over time. This is controlled by the hyperparameters β_1 and β_2, which determine how much weight is given to past gradients and squared gradients, respectively. In PyTorch Adam implementation they are equal $\beta_1 = 0.9$ and $\beta_2 = 0.999$ (PyTorch Vision Team, n.d.-a). Bellow is shown an accumulation of inertia with exponential smoothing:

$$h_t^j = \frac{\beta_1 \cdot h_{t-1}^j + (1 - \beta_1) \cdot g_{tj}}{1 - \beta_1^t} \tag{12}$$

Dynamically adjust learning rate:

$$G_t^j = \frac{\beta_2 \cdot G_{t-1}^j + (1 - \beta_2) \cdot g_{tj}^2}{1 - \beta_2^t} \tag{13}$$

Combination of them:

$$w_t^j = w_{t-1}^j - \frac{\eta_t}{\sqrt{G_t^j + \epsilon}} \cdot h_t^j \tag{14}$$

[10]RMSprop - Root Mean Squared Propagation

The performance of mentioned algorithms is illustrated in Figure 27, which shows that Adam achieves superior results when training neural networks on the MNIST dataset.

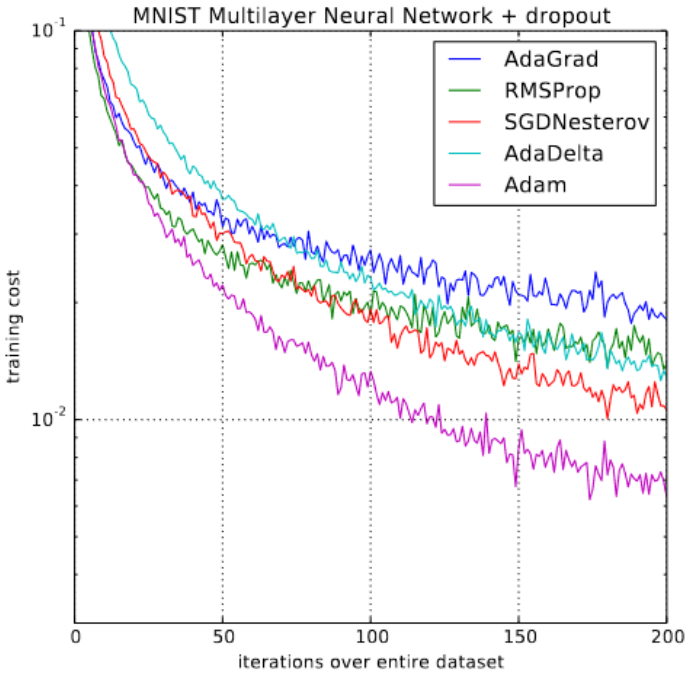


Figure 27: Training of multilayer neural networks on MNIST images. Neural networks using dropout stochastic regularization. Source: (Kingma & Ba, 2017)

Adam's balance between fast convergence and adaptive learning rate adjustments makes it effective for complex models, even in challenging optimization landscapes. Also, Adam (or AdamW with an additional regularization feature) would be used in further practical part as an optimizer.

4.4 Loss Function

The main goal of training a neural network is to optimize a loss function or find an optimal local minimum with an optimizer. Gradient descent, the chain rule, and the back-propagation algorithm allow for the calculation of derivatives and the adjustment of the network's weights to minimize the loss function. The loss function visualization is possible with certain caveats and limitations, as Li et al. (2018b) highlighted challenges such as non-convexity and high dimensionality.

Skip connections stand for mitigating the vanishing gradient problem in deep architectures by creating "shortcuts" that allow gradients to flow through the net. ConvNeXt is inspired by ResNeXt and incorporates bottleneck blocks (Liu et al., 2022), so it also uses residual blocks. Figure 28 demonstrates the impact of skip connections on the loss surface in ResNet-56. The surface without skip connections (a) is highly irregular, containing numerous sharp minima and narrow paths that can trap the optimizer in suboptimal points, making convergence slower and less stable. In contrast, the loss surface with skip connections (b) is considerably smoother, providing a more direct path toward the minimum and enabling more efficient optimization.

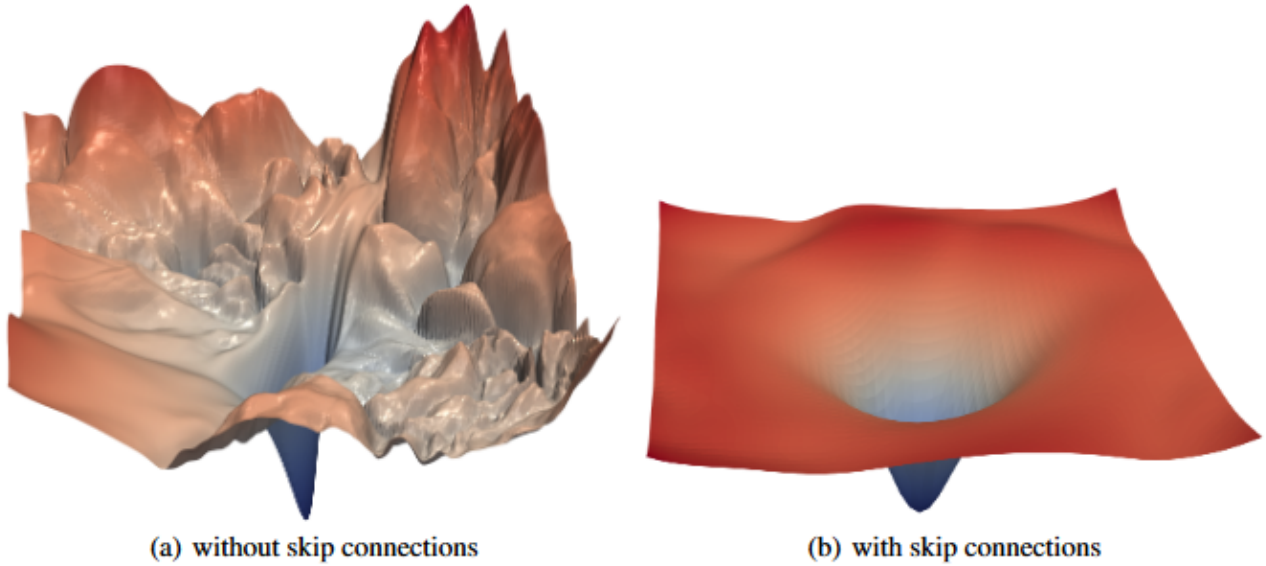


Figure 28: The loss surfaces of ResNet-56 with/without skip connections. Source: (Li et al., 2018a)

4.5 Regularization

Due to the non-convexity of the loss function, neural networks are prone to overfitting, especially when the model complexity is high, or the training data is limited. Additionally, modern models tend to be highly hyperparameterized: the number of observations (n) is often much smaller than the number of parameters (k). For example, ConvNeXt small model has 50,223,688 parameters according to Pytorch documentation (PyTorch Vision Team, 2021). As a result, Fisher's asymptotic properties (Fisher, 1922), where model performance improves as $n \to \infty$ no longer apply in their original form and must be reconsidered in the context of $\frac{n}{k} \to \infty$, where the ratio of observations to model parameters tends to infinity.

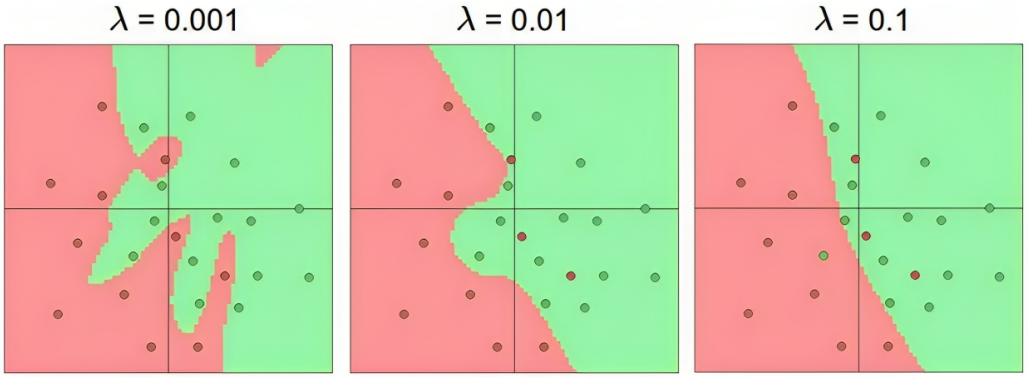


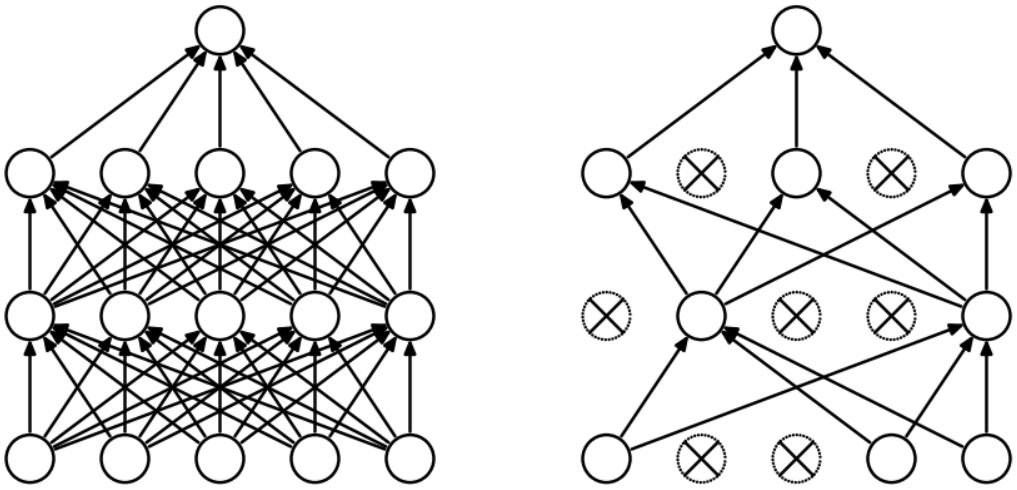
Figure 29: The effects of regularization strength: Each neural network above has 20 hidden neurons, but changing the regularization strength makes its final decision regions smoother with a higher regularization (Stanford University, 2017).

Classical machine learning regularization $L_1 = \lambda \cdot \sum_i w_i$ and $L_2 = \lambda \cdot \sum_i w_i^2$, where λ is a penalty coefficient, is not used in neural network models. However, deep learning has developed its own methods that provide a similar or equivalent regularizing effect. As discussed in the book

by Goodfellow et al. (2016, p. 249) they found the effect of Early Stopping method comparable to L_2-regularization. Specifically, trajectory of length τ (the number of iterations before training is halted) ends at a point that corresponds to a minimum of the L_2-regularized objective. Additionally, the following sections will discuss Dropout and Weight Decay as regularization methods, alongside with Early Stopping.

4.5.1 Dropout

The key idea is to randomly drop units (along with their connections) from the neural network during training. This prevents units from co-adapting too much (Srivastava et al., 2014). In other words, dropout is a regularization technique to make neurons more resilient and independent by randomly resetting the output of a neuron during forward pass with a chance of p. Figure 30 demonstrates a standard neural net with 2 hidden layers on the left side and an example of a thinned net produced by applying dropout to the network on the left (Srivastava et al., 2014).



(a) Standard Neural Net (b) After applying dropout.

Figure 30: Dropout Neural Net Model (Srivastava et al., 2014)

For any layer l, $r^{(l)}$ is a vector of independent Bernoulli random variables each of which has probability p of being 1 (Srivastava et al., 2014). During the forward pass in the training phase, the output *(o)* of each neuron is calculated as:

$$o = d \cdot f(X \cdot W) \tag{15}$$

where d is a binary dropout mask generated from $r^{(l)}$ and $f(X \cdot W)$ represents the raw output of the layer without regularization applied. To ensure that the expected output remains the same during training and testing, the outputs are scaled by a factor of $\frac{1}{1-p}$ during training (PyTorch Vision Team, n.d.-b):

$$o = \frac{1}{1-p} \cdot d \cdot f(X \cdot W) \tag{16}$$

During the testing phase, dropout is not applied, and the network uses the full set of neurons without masking:

$$o = f(X \cdot W) \tag{17}$$

Thanks to this method, we obtain an ensemble from a set of subsets of neurons. A neural net with n units, can be seen as a collection of 2^n possible thinned neural networks. These networks all share weights so that the total number of parameters is still $O(n^2)$, or less (Srivastava et al., 2014). Also, (Wager et al., 2013) mentioned, that for generalized linear models, dropout performs a form of adaptive regularization. Using this viewpoint, we show that the dropout regularizer is first-order equivalent to an L_2 regularizer applied after scaling the features by an estimate of the inverse diagonal Fisher information matrix (Wager et al., 2013).

4.5.2 Weight Decay

Another method to prevent overfitting is *weight decay* regularization. Weight decay penalizes large weights by directly reducing their magnitude during optimization, encouraging simpler models with better generalization properties. This concept is particularly powerful when combined with adaptive gradient optimizers like Adam, resulting in the *AdamW* optimizer.

AdamW decouples weight decay from the gradient-based updates, addressing issues present in traditional L_2 regularization when used with adaptive algorithms. The parameter update rule for AdamW is shown bellow, where λ is weight decay coefficient that controls the strength of regularization, G_t is accumulated squared gradients and h_t is accumulated momentum:

$$w_t = (1 - \eta \cdot \lambda) \cdot w_{t-1} - \frac{\eta_t}{\sqrt{G_t} + \epsilon} \cdot h_t \tag{18}$$

For standard stochastic gradient descent (SGD), L_2 regularization and weight decay are mathematically equivalent when scaled by the learning rate. However, this equivalence breaks down for adaptive gradient algorithms like Adam. As highlighted by Ilya and Frank (2019), Adam's adaptive nature, which relies on variance in gradient magnitudes, renders L_2 regularization less predictable and prone to unintended behavior.

L_2 regularization is not effective in Adam. One possible explanation why Adam and other adaptive gradient methods might be outperformed by SGD with momentum is that common deep learning libraries only implement L_2 regularization, not the original weight decay. Therefore, on tasks/datasets where the use of L_2 regularization is beneficial for SGD (e.g., on many popular image classification datasets), Adam leads to worse results than SGD with momentum (for which L_2 regularization behaves as expected). (Ilya & Frank, 2019).

Moreover, the effectiveness of weight decay depends on the total number of weight updates. Empirical studies suggest that the optimal weight decay coefficient decreases as the number of batch passes or runtime increases (Ilya & Frank, 2019). This observation underscores the need for dynamic tuning of weight decay based on task-specific training dynamics and model requirements.

Figure 31 compares the performance of Adam and AdamW on CIFAR-10 with a 262x96d ResNet architecture. The top-left plot shows that Adam struggles with stability at higher weight decay values, while AdamW maintains steady convergence. The top-right plot highlights Adam's erratic test error compared to AdamW's consistent improvement. The bottom-left plot shows that AdamW achieves lower test error across most weight decay settings, while the bottom-right plot confirms AdamW's superior generalization, achieving lower test error for a given training loss.

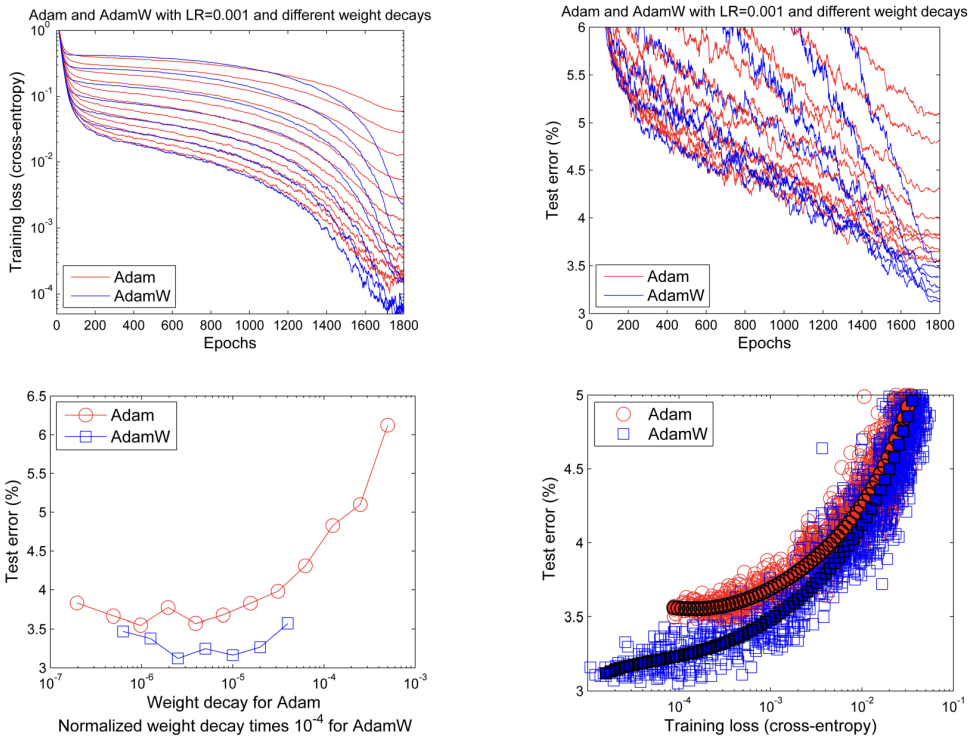


Figure 31: Learning curves (top row) and generalization results (bottom row) obtained by a 262x96d ResNet trained with Adam and AdamW on CIFAR-10 (Ilya & Frank, 2019).

These results highlight the importance of decoupling weight decay from gradient updates, as done in AdamW, to improve stability and generalization performance.

4.5.3 Epochs and Early Stopping

There is no universal rule for setting optimal numbers of epochs, but it's recommended to start on small numbers and increase them as neural network performance grows. Another useful instrument is early stopping, which saves the best weights of the network based on the minimal value of the loss function. This technique allows us to automatically stop training when overfitting has begun, and validation loss is about to increase. There's some problem - validation loss can be very volatile, and we don't want to stop training if the loss is higher on a small number than on the previous epoch (see Figure 32), so we need to set some patience - it might help to stop training at the right moment because several epochs without decreasing loss on validation phase indicate overfitting or inability to find a better minimum due to high or low learning rate, it is also called divergence.

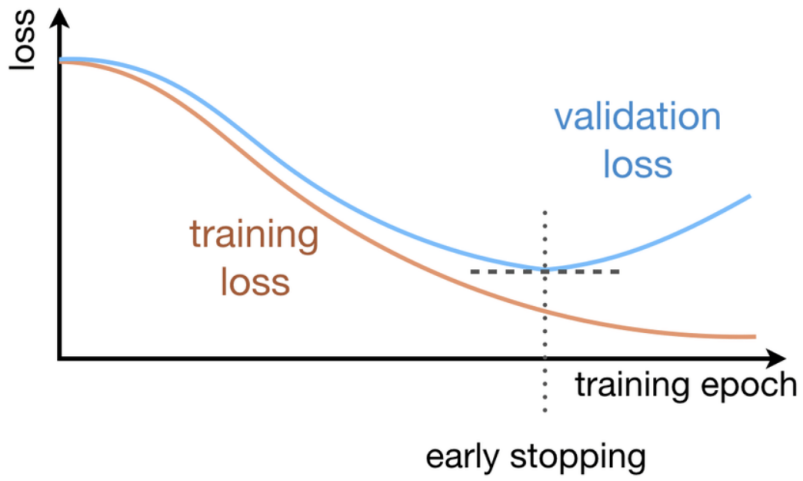


Figure 32: Stop learning when loss starting increase (Stanford University, 2017).

4.5.4 Regularization - Summary

The chapter reviewed key regularization techniques: dropout, weight decay, and early stopping that address the overfitting risks inherent in neural nets, especially given the high parameter-to-observation ratio in modern models. Dropout, by randomly omitting neurons during training, creates an ensemble-like effect that promotes neuron independence. Weight decay regularization penalizes large weights to maintain model generalization. Early stopping preserves optimal weights, stops training when validation loss no longer improves and further train cycles doesn't make any sense.

4.6 Summary

This chapter explored several key techniques for hyperparameter tuning and regularization, emphasizing their relevance to modern deep neural networks. Advanced activation functions, such as GELU, will be used for the ConvNeXt classifier due to their superior performance on datasets like CIFAR-10 and ImageNet. The use of dynamic learning rate schedulers, specifically *CosineAnnealingLR*, will improve efficient training by adapting the learning rate to improve convergence and explore multiple local minima. For optimization, Adam and AdamW versions will be utilized. Regularization methods, including dropout, weight decay, and custom early stopping mechanism with adjustable value of tolerance, will be integrated to mitigate overfitting. I will be using primary PyTorch's implementation of mentioned methods.

5 Model training

5.1 Instruments and setup

First of all, data should be prepared and adequately transformed according to the neural network's architecture needs. This paper will mainly use a PyTorch library to import models from Pytorch's repository. Also they provide optimizers, loss functions (or criterion), learning rate schedulers, interface to transfer calculation from CPU to GPU with CUDA[11] and many image transformation tools like resize, crop and normalization with an output of native high-dimensional PyTorch object called tensor. WANDB and tensorboard will be used for experiments logging. Finally, the last key library will be sci-kit-learn to divide data for training and validation sets and also shuffle it to prevent overfitting by memorizing an order of data.

5.2 Data preparation

5.2.1 Data preparation pipeline

The final dataset was created using a custom data preparation pipeline, as illustrated in Figure 33.

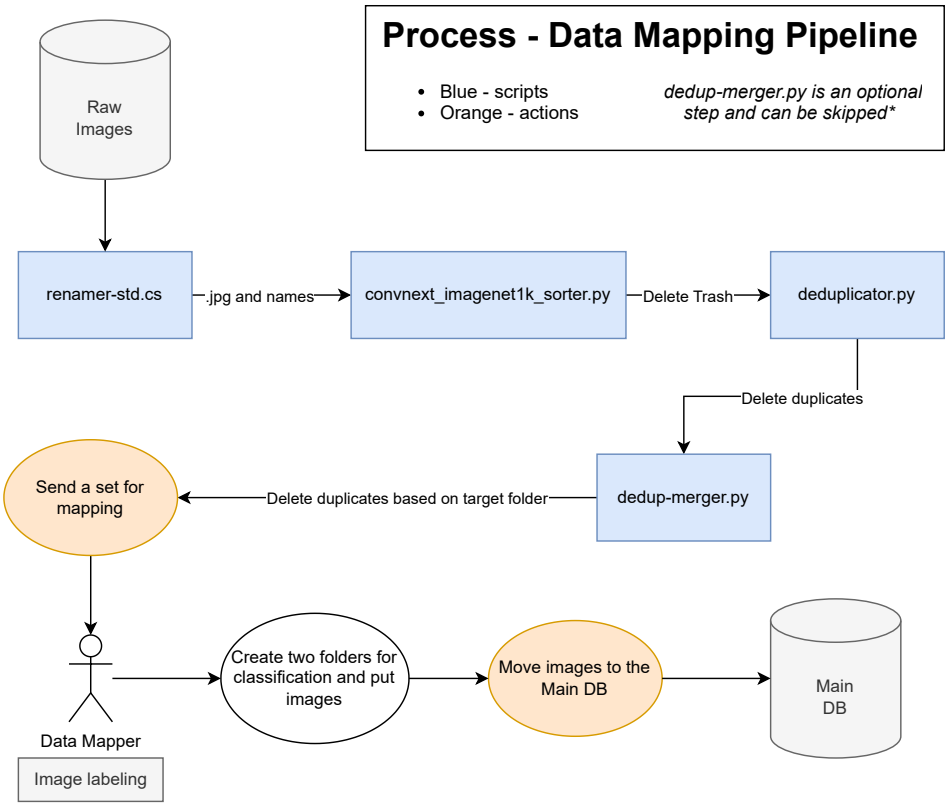


Figure 33: Data preparation pipeline (Author, 2023)

[11]Compute Unified Device Architecture - a software layer that gives direct access to the GPU's virtual instruction set and parallel computational elements for the execution of compute kernels.

For each step in blue rectangle individual custom Python or C# script was created to execute a specific step in the preparation process. The pipeline performs several key operations to ensure the data is standardized, organized, and ready for model training.

The process begins with renaming and standardization. This step ensures consistent naming conventions and file formats across the dataset. Using the script from (Ivan, 2024), images are systematically renamed following a structured naming scheme, such as *light_ 3.jpg*, where files are numbered incrementally. Script also convert all images to a standard JPG format, to ensure compatibility with the neural network.

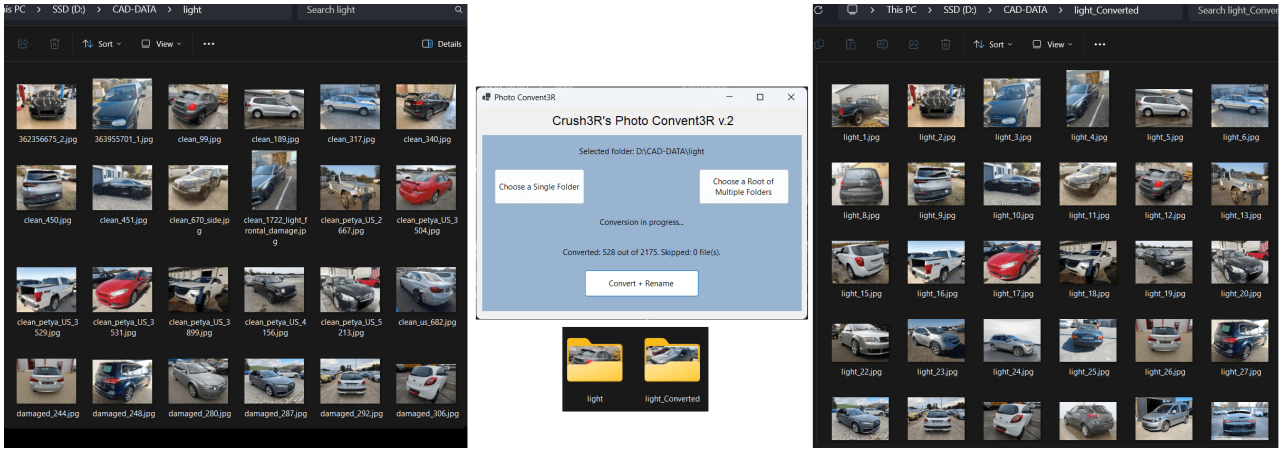


Figure 34: Renaming script from (Ivan, 2024) in action. Left side - before processing, right side - after processing (Author, 2024).

Next, the sorting and classification script organizes images by grouping them according to their classes and target labels based on predefined criteria derived from the ImageNet dataset (VisionLab, 2009). Removes all seats, wheels, etc., and keep only vehicle-related objects from ImageNet classes: *amphibian, convertible, recreational vehicle, ashcan, police van, sports car, moving van, racer, tank, car mirror, cab, tow truck, minibus, half track, minivan, ambulance, grille, limousine, radiator, jeep, motor scooter, beach wagon, car wheel, snowmobile, pickup.*

```
for file_name in tqdm(os.listdir(image_dir)):
    if file_name.endswith('.jpg'):
        image_path = os.path.join(image_dir, file_name)
        try:
            predicted_label = get_label(image_path)
            deleted_wrong_label(predicted_label, image_path, delete=True)
        except RuntimeError or OSError:
            pass
    else:
        assert image_path, 'image_path is None'
        with open('skipped.txt', 'a') as f:
            f.write(image_path + '\n')
```

Listing 1: Image processing function that iterates over files in a folder, predicts labels, and removes incorrectly labeled images (Author, 2023).

The next step is deduplication to remove duplicate images because scrapped images were commonly reused on different auction platforms. This process uses a combination of the Pillow library and MD5[12] hashing to generate a unique hash for each image based on its pixel values. This hash serves as an identifier, enabling the detection and removal of duplicate images even if they have different file names. The following code snippet shows the deduplication function that generates an MD5 hash for each image:

```python
def get_hash(image_path: str) -> str | None:
    image = Image.open(image_path)
    try:
        image = image.resize((10, 10), Image.LANCZOS)
    except OSError:
        print(f'Could not process image: {image_path}')
        return None
    grey_image = image.convert('L')  # Convert the image to grayscale
    pixel_array = np.array(grey_image).flatten()  # 1D array of pixels
    hashed = hashlib.md5(pixel_array).hexdigest()
    return hashed
```

Listing 2: Function to generate MD5 hash for deduplication (Author, 2023).

Once duplicates are removed, the data enters a manual mapping phase. In this step, the remaining images are reviewed and manually mapped to their appropriate categories, ensuring that each image is accurately labeled. After the mapping phase, the final processed data is ready for integration into the main dataset.

5.2.2 Data splitting

A crucial part of preparing data for models is ensuring a proper split between training, validation, and test sets. First, from the entire dataset, a fixed subset of 2,000 images (1,000 per class) is extracted and reserved as the test set, which will be used exclusively to evaluate the final models and compare them. This ensures that the test set remains unseen during the training and validation phases, providing an unbiased evaluation of the model's performance. From the remaining data, the dataset is split into training and validation subsets in an 80-20 ratio. This is defined by a constant VAL_SIZE for use in the $train\_test\_split$ function from the *scikit-learn* package. A fixed random seed (e.g., $random\_state = 42$) is applied to ensure reproducibility, guaranteeing the same split across multiple runs and no overlap between training and validation subsets. The listing below shows the custom *CadDatasetV2* wrapper extends PyTorch's Dataset class to provide enhanced functionality, including automatic class-balanced train-validation splitting, flexible memory management with optional preloading, and out-of-the-box integration of transformations. In the further section, this class will be used as a source of data for the PyTorch *DataLoader* class.

```python
from torch.utils.data import Dataset
from torchvision import transforms as T
```

[12]MD5 is a hash function that generates a unique identifier for data, often used to detect duplicate files.

```
from sklearn.model_selection import train_test_split
class CadDatasetV2(Dataset):
    SPLIT_RANDOM_SEED = 42
    TEST_SIZE = 0.20

    def __init__(self, root, train=True, load_to_ram=True, transform=None):
        super().__init__()
        self.root = root
        self.train = train
        self.load_to_ram = load_to_ram
        self.transform = transform
        self.to_tensor = T.ToTensor()
        self.all_files = []
        self.all_labels = []
        self.images = []
        self.classes = sorted(os.listdir(self.root))
        for i, class_name in tqdm(enumerate(self.classes), total=len(self.
    classes)):
            files = sorted(os.listdir(os.path.join(self.root, class_name)))
            train_files, val_files = train_test_split(files, random_state=
    self.SPLIT_RANDOM_SEED + i,test_size=self.VAL_SIZE)
            if self.train:
                self.all_files += train_files
                self.all_labels += [i] * len(train_files)
                if self.load_to_ram:
                    self.images += self._load_images(train_files, i)
            else:
                self.all_files += val_files
                self.all_labels += [i] * len(val_files)
                if self.load_to_ram:
                    self.images += self._load_images(val_files, i)

    def _load_images(self, image_files, label):
        images = []
        for filename in image_files:
            image = Image.open(os.path.join(self.root, self.classes[label],
    filename)).convert('RGB')
            images += [image]
        return images

    def __getitem__(self, item):
        label = self.all_labels[item]
        if self.load_to_ram:
            image = self.images[item]
        else:
            filename = self.all_files[item]
            image = Image.open(os.path.join(self.root, self.classes[label],
    filename)).convert('RGB')
        if self.transform:
```

```
            image = self.transform(image)
        return image, label
```

Listing 3: Custom more functional wrapper for Pytorch Dataset class (Author, 2024)

5.2.3 Data transformation

As was already said, the paper would use pre-trained models on the ImageNet dataset with 1,000 classes at a resolution of 224x224 pixels for each image. Additionally, standard ImageNet normalization would be applied to the input data; here's an implementation of it:

```
from torchvision import transforms as T

transform = T.Compose([
      T.Resize((224, 224)),
      T.ToTensor(),
      T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
  ])
```

Listing 4: Image transformations with Pytorch library (Author, 2023)

Vehicles are more likely to have damage on the front or other prominent areas such as forward and rare parts. Using a single resizing instead of resizing to 256 pixels square with following 224 pixels center cropping as default ImageNet transformation helps preserve these critical features, especially on side photos, as illustrated in Figure 36 and 35 by comparing these two techniques:

Figure 35: Single resize is applied (Author, 2023)

Figure 36: Resize with following center crop (Author, 2023)

The final data preparation phase involves creating DataLoaders, which manage data loading from disk to memory during model training and validation. The DataLoader in PyTorch can load batches of data, handle shuffling, and parallelize data loading with multiple workers. There are several differences between train and validation loaders. First, a validation batch size is two times bigger due to the missing back-propagation phase, unlike in training one, which allows processing more photos in one batch to speed up the whole process and also uses GPU's memory at full. Shuffle is used in the train to prevent overfitting by memorizing an order of images, and it is useless at the validation phase by its nature. The pin_memory parameter speeds up the data transfer between CPU and GPU. Here's a python code for this part:

```
from utils.utils import CadDatasetV2, transform, EarlyStopper
from torch.utils.data import DataLoader

train_dataset = CadDatasetV2(root='data/HC_5000', train=True,
                             load_to_ram=False,
                             transform=transform)

val_dataset = CadDatasetV2(root='data/HC_5000', train=False,
                           load_to_ram=False,
                           transform=transform)

train_dataloader = DataLoader(train_dataset, batch_size=32, shuffle=True,
 pin_memory=True, num_workers=1)

val_dataloader = DataLoader(val_dataset, batch_size=64, shuffle=False,
 pin_memory=True, num_workers=1)
```

Listing 5: Creating DataLoaders for training and validation datasets (Author 2024)

5.3 Training process

There are two neural network architectures to be benchmarked: ResNet-56 and ConvNeXt-tiny. Each model will follow an identical training flow: an optimal learning rate will first be determined using a learning rate finder. Then, different regularization techniques will be evaluated, including no regularization, Adam optimizer with dropout, and AdamW optimizer. The performance of each model-regularization combination will be assessed based on key metrics: validation accuracy, F1 score and final model size. Training accuracy would not be included because every tested model shows it around 97-99%. Graphs will be generated for each combination to illustrate metric comparisons across models.

The initial hyperparameter search is conducted on a subset of 10,000 images (5,000 per class) to identify the best-performing configurations for each architecture. Once the two optimal configurations are determined, these models are trained on the entire train-validation dataset (21,000 images per class). The final evaluation of their performance and generalization capabilities will be conducted using metrics obtained from an independent test dataset, ensuring an unbiased comparison of the selected configurations.

5.3.1 Transfer learning or Fine-tuning

Both transfer learning and fine-tuning are techniques used to leverage pre-trained models on new tasks. Transfer learning involves using a pre-trained model directly as a feature extractor, with the original model layers frozen[13] and only the final layers retrained on new data. Fine-tuning, on the other hand, entails unfreezing some or all of the pre-trained layers, allowing them to be adjusted based on the new dataset, which should lead to better adaptation to specific tasks.

After initial tests with the stock version of the ConvNeXt-tiny model, fine-tuning proved to be the more effective approach, as transfer learning has inefficient accuracy. Fine-tuning involves unfreezing and updating parameters in deeper layers of the pre-trained model, enabling it to better adapt to the nuances of the target dataset and making it the better choice in my scenario. The graphs below show superior performance of the fine-tuning approach, even from the initial epochs.

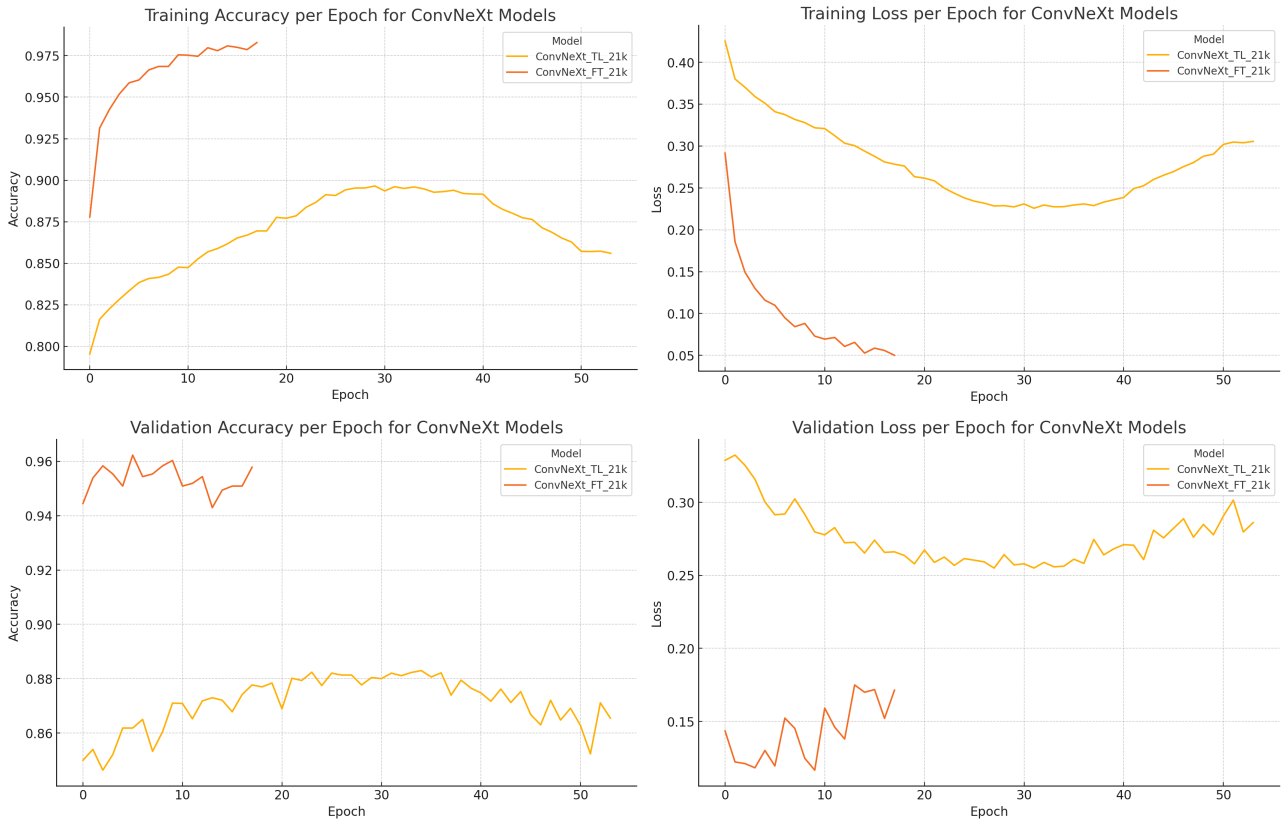


Figure 37: Metrics comparison between transfer learning (yellow) and fine-tuning (orange) (Author, 2024)

5.3.2 Training Algorithm

The identical training algorithm will be used across all models to ensure consistency and comparability of results. The training pipeline is implemented as a composition of multiple modular functions, each addressing a distinct aspect of the training and evaluation process. Below is

[13]In frozen layers, gradients are not computed during the backpropagation phase

a detailed description of these components: a metric for evaluating model performance during training and validation is accuracy. The following function computes the accuracy by comparing predicted labels with the ground truth. This function utilizes the *torch* and *numpy* libraries for efficient tensor operations and array manipulations:

```
def get_accuracy(output, target_labels) -> float:
  pred = torch.argmax(output, dim=1).cpu().numpy()
  acc = np.mean(pred == target_labels.cpu().numpy())
  return acc
```

Listing 6: Function to calculate accuracy (Author 2024)

The training process is divided into *train* and *validation* phases, which are executed iteratively over the dataset. Each phase is encapsulated in dedicated functions to ensure modularity and reusability. Training Epoch Function manages the forward pass, backward pass, and parameter updates for each mini-batch of data. The model is set to training mode, enabling features like dropout and batch normalization. Accuracy and loss metrics are recorded for each batch.

```
def train_epoch(model: nn.Module,
                optimizer: Optimizer,
                criterion: CrossEntropyLoss,
                train_loader: DataLoader,
                device: str = "cuda:0") -> tuple:
  loss_log, acc_log = [], []

  model.train() # Set model to training mode activate training layers
 Dropout & BatchNorm
  for batch_num, (inputs, labels) in enumerate(train_loader):
      # inputs: [batch_size x num_channels x height x width]
      inputs, labels = inputs.to(device), labels.to(device)
      optimizer.zero_grad() # Zero the parameter gradients
      outputs = model(inputs) # outputs: [batch_size x num_classes]
      loss = criterion(outputs, labels) # Forward pass
      loss.backward() # Backward pass and optimization
      optimizer.step()
      loss_log.append(loss.item())
      acc = get_accuracy(outputs, labels)  # Calculate accuracy
      acc_log.append(acc)

  return loss_log, acc_log
```

Listing 7: Function to perform train epoch (Author 2024)

The validation epoch function evaluates the model on the validation set without updating the model's parameters. Gradient computation is disabled to reduce computational overhead during validation.

```
@torch.no_grad()
def validate_epoch(model: nn.Module,
                   criterion: CrossEntropyLoss,
                   val_loader: DataLoader,
```

```
                    device: str = "cuda:0"
                    ) -> tuple:
    loss_log, acc_log = [], []
    model.eval()
    for batch_num, (val_inputs, val_labels) in enumerate(val_loader):
        val_inputs, val_labels = val_inputs.to(device), val_labels.to(device
    )
        val_outputs = model(val_inputs)
        val_loss = criterion(val_outputs, val_labels)
        acc = get_accuracy(val_outputs, val_labels)
        acc_log.append(acc)
        val_loss = val_loss.item()
        loss_log.append(val_loss)

    return loss_log, acc_log
```

Listing 8: Function to perform validation epoch without gradient calculation (Author 2024)

The central training logic is implemented in the train function, which integrates the aforementioned modular components. This function orchestrates the training loop, implements learning rate scheduling, early stopping, and logs training progress using tools like TensorBoard.

```
def train(model: nn.Module,
          optimizer: Optimizer,
          criterion: CrossEntropyLoss,
          train_loader: DataLoader,
          val_loader: DataLoader,
          num_epochs: int,
          early_stopper: EarlyStopper,
          experiment_name: str,
          device: str = "cuda:0",
          schedule: LRScheduler = None) -> nn.Module:
    logger = TensorBoardLogger(experiment_name=experiment_name)
    model.to(device)
    for epoch in tqdm(range(num_epochs)):
        train_loss_log, train_acc_log = train_epoch(model, optimizer,
    criterion, train_loader)
        val_loss_log, val_acc_log = validate_epoch(model, criterion,
    val_loader)

        if schedule is not None: # Scheduler step
            schedule.step()

        train_loss = np.mean(train_loss_log) # Metrics
        train_acc = np.mean(train_acc_log)
        val_loss = np.mean(val_loss_log)
        val_acc = np.mean(val_acc_log)

        logger.log_metrics(train_loss, train_acc, val_loss, val_acc, epoch)
        wandb.log({"acc": val_acc, "loss": val_loss}) # log metrics to wandb
```

```
27
28          if early_stopper.early_stop(np.mean(val_loss_log), model):
29              print(f'Early stopping on epoch {epoch}')
30              model.load_state_dict(early_stopper.best_weights)
31              break
32
33      logger.close()
34      wandb.finish()
35      return model
```

Listing 9: Main model train function (Author 2024)

5.4 ResNet56

The first model selected for benchmarking is the ResNet56, a variant of the ResNet architecture developed by Microsoft in 2015.

5.4.1 ResNet56 - Stock

To begin the benchmarking process, we start with a "stock" ResNet56 model, using the pre-trained weights available in the Torch library. The initial phase involves finding the optimal learning rate. First, import the ResNet model and adapt the final fully connected layer to handle only two classes: clean and damaged.

```
1   from torchvision.models import ResNet50_Weights, resnet50
2
3   model = resnet50(weights=ResNet50_Weights.DEFAULT)
4   num_ftrs = model.fc.in_features
5   model.fc = nn.Sequential(
6       nn.Linear(num_ftrs, 2)
7   )
```

Listing 10: Import the ResNet model from torch repository (Author, 2024)

Next, configure the default Adam optimizer, cross-entropy loss function, and perform a learning rate range test using the LRFinder (David Silva, Nale Raphael, 2020) package.

```
1   from torch_lr_finder import LRFinder
2   from torch import optim, nn
3
4   loss_func = nn.CrossEntropyLoss()
5   optimizer = optim.Adam(model.parameters(), lr=1E-7)
6   lr_finder = LRFinder(model, optimizer, loss_func, device="cuda")
7   lr_finder.range_test(train_dataloader, end_lr=0.01, num_iter=100)
8   lr_finder.plot() # to inspect the loss-learning rate graph
```

Listing 11: Using the LRFinder (Author, 2024)

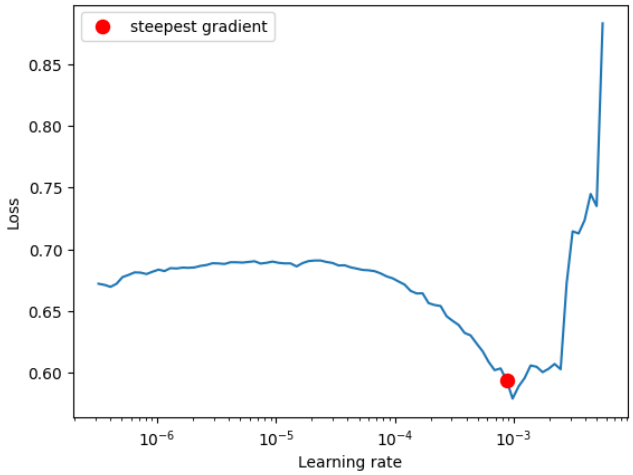


Figure 38: Loss function changes of ResNet50 depends on learning rates, highlighting the steepest gradient (Author, 2024)

The steepest slope was identified at a learning rate of $8.70e-4$. After establishing an optimal learning rate, continue with training the ResNet56 model by executing a main train script. The training process incorporates two key techniques to enhance performance and prevent overfitting. First, the *CosineAnnealingLR* scheduler, described in the 4.2.1, is used to dynamically adjust the learning rate during training. This helps the optimizer escape potential local minima. Second, early stopping is implemented as a regularization technique to halt training when validation performance stagnates, preventing overfitting to the training data. The early stopping mechanism is configured with a patience of 15 epochs and a minimum validation loss improvement threshold of 0.01.

```
loss_func = nn.CrossEntropyLoss()
opt = optim.Adam(model.parameters(), lr=LR)
scheduler = optim.lr_scheduler.CosineAnnealingLR(opt, EPOCHS)
early_stopper = EarlyStopper(patience=15, min_delta=0.01, verbose=True)

trained_model = train(model, opt,
                      loss_func,
                      train_dataloader,
                      val_dataloader,
                      EPOCHS,
                      experiment_name=EXPERIMENT_NAME,
                      schedule=scheduler,
                      early_stopper=early_stopper)
```

Listing 12: Training function for Resnet56 (Author, 2024)

The results obtained from training the stock ResNet50 model, as visualized in the accuracy and loss graphs (Figure 39).

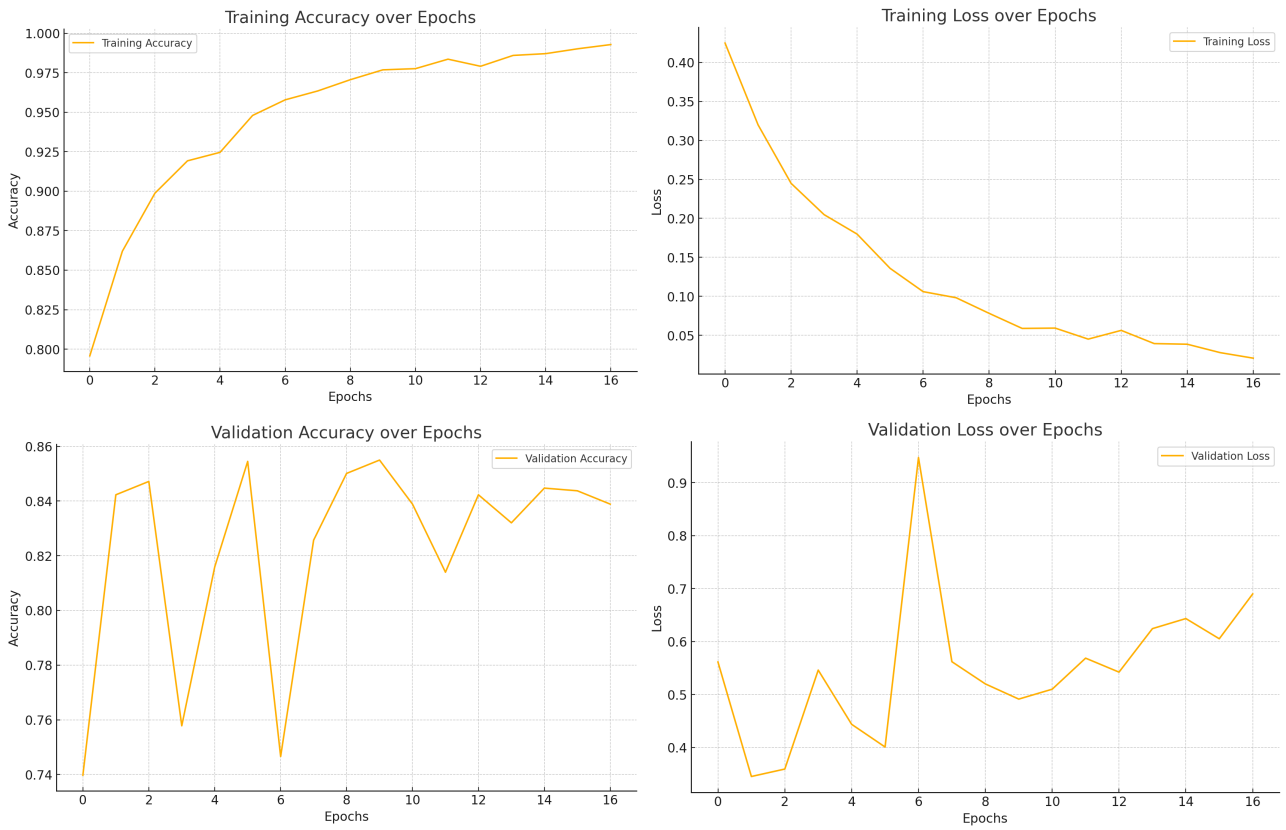


Figure 39: Training and Validation Metrics for Resnet50 without regularization (Author, 2024)

Graphs show signs of overfitting, in the training accuracy graph (top left), we observe a rapid and consistent increase, eventually nearing 100% accuracy for 16th epoch. Simultaneously, the training loss (top right) shows a steady decline, which indicates that the model is effectively learning the provided data. However, on validation graphs a different pattern is presented. The validation accuracy (bottom left) fluctuates significantly and stabilizes at a lower value compared to the training accuracy, never reaching the same peak levels. Similarly, the validation loss (bottom right) displays irregular behavior with frequent increases, indicating that the model's ability to generalize unseen data is limited. With given observations, task is required some regularization techniques. In the following section, dropout layer will be added to the classifier and analyze its impact on the model's performance.

Despite obvious overfiiting, model will be benchmarked on the final third test dataset to calculate metrics and create confusion matrix for the further comparison among regularized ones.

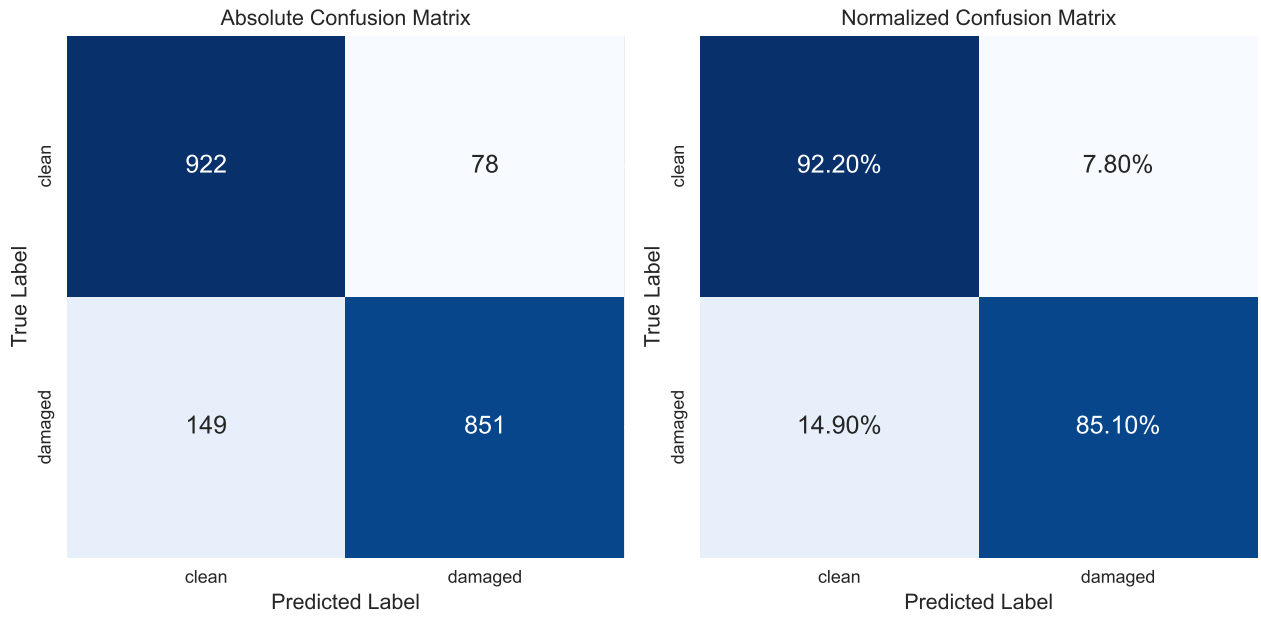


Figure 40: Confusion Matrix for ResNet50 - Test Dataset Performance (Author, 2024)

Table 2: Metrics for stock Resnet50 model (Author, 2024)

Model	Dataset size	Val Accuracy	Test Accuracy	Test F1 Score
ResNet50	10,000	0.851	0.886	0.882

5.4.2 ResNet56 - Dropout

To improve generalization and reduce overfitting, a dropout regularization layer is introduced into the model architecture. This layer randomly deactivates neurons during training with a probability of $p = 0.5$. Only one new line is added to model's classifier:

```
from torchvision.models import ResNet50_Weights, resnet50

model = resnet50(weights=ResNet50_Weights.DEFAULT)
num_ftrs = model.fc.in_features
model.fc = nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(num_ftrs, 2)
)
```

Listing 13: Add the Dropout layer to the classifier (Author, 2024)

The steepest slope was identified at a learning rate of $8.70e-4$ (see Figure 41), same as before without regularization. It happened because the dropout doesn't significantly change the landscape of loss function.

Despite the addition of dropout regularization with a probability of $p = 0.5$, overfitting signs persist in the ResNet50 model, as shown in Figure 42. Although the dropout layer did slightly improve validation accuracy and test metrics (see Table 3 and Figure 43), they are

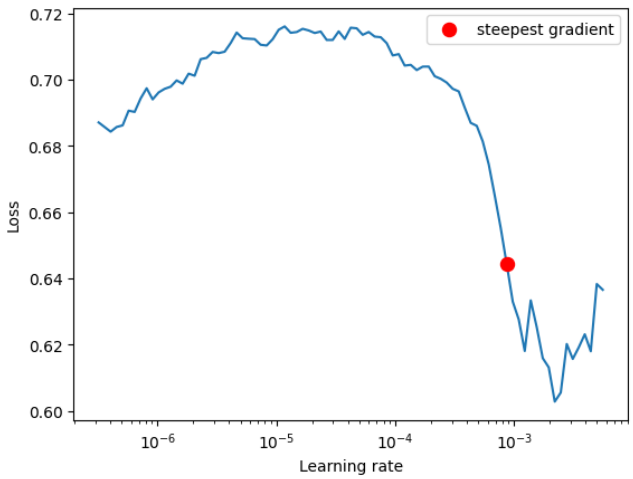


Figure 41: Loss function changes of ResNet50 with Dropout depends on learning rates, highlighting the steepest gradient (Author, 2024)

marginal and almost not visible. Given the negligible impact of dropout in this scenario and the relatively simple modifications made to the fully connected layer, this leads to the conclusion that the dropout technique is ineffective in addressing overfitting for this specific model setup. Therefore, dropout regularization will not be used in further models throughout this study. Instead, alternative regularization techniques, such as weight decay, will be explored to prevent overfitting.

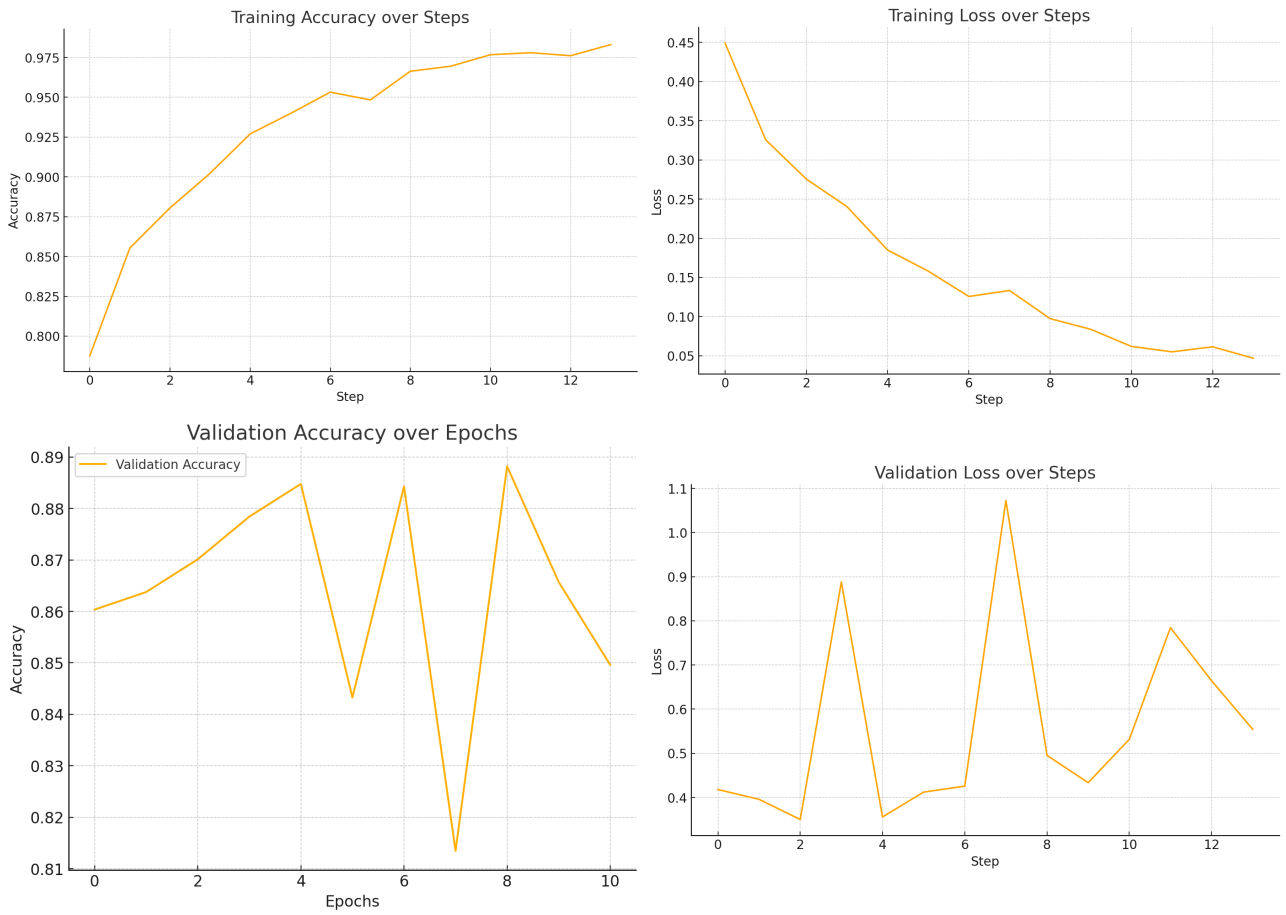


Figure 42: Training and Validation Metrics for Resnet50 with Dropout ($p = 0.5$) regularization (Author, 2024)

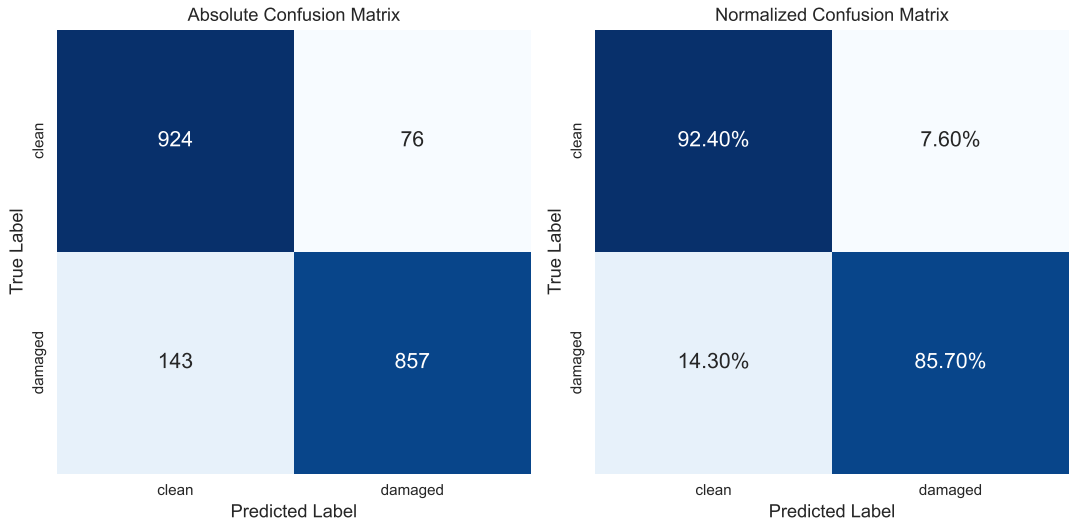


Figure 43: Confusion Matrix for ResNet50 with Dropout regularization - Test Dataset Performance (Author, 2024)

Table 3: Metrics for Resnet50 with Dropout regularization (Author, 2024)

Model	Dataset size	Val Accuracy	Test Accuracy	Test F1 Score
ResNet50 Dropout	10,000	0.853	0.890	0.887

5.4.3 ResNet56 - AdamW

The last model from ResNet family would use the weight decay regularization with decay value of 0.01. AdamW integrates weight decay directly into the optimization process, reducing overfitting by penalizing large weights. This is done by adding the following property to AdamW optimizer from PyTorch:

```
opt = optim.AdamW(model.parameters(), lr=LR, weight_decay=0.01)
```

Listing 14: Add the weight decay to the optimizer (Author, 2024)

The learning rate yielded from lr-finder is $3.85e-4$, that is different from previous models with $8.7e-4$. Overall, the AdamW optimizer with weight decay improves training stability

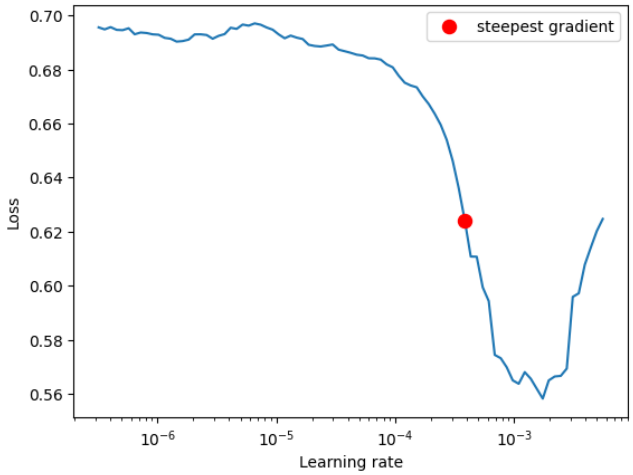


Figure 44: Loss function changes of ResNet50 with AdamW depends on learning rates, highlighting the steepest gradient (Author, 2024)

and generalization to a certain extent, as reflected by the relatively high validation accuracy. Importantly, this configuration achieved the highest metrics observed so far, with a validation accuracy of 88.2%, test accuracy of 90.3%, and an F1 score of 0.896 (see Table 4). These results indicate that weight decay provides a significant improvement in the model's ability to generalize to unseen data. However, the fluctuations in the validation curves suggest that there might still be some space for further enhancement.

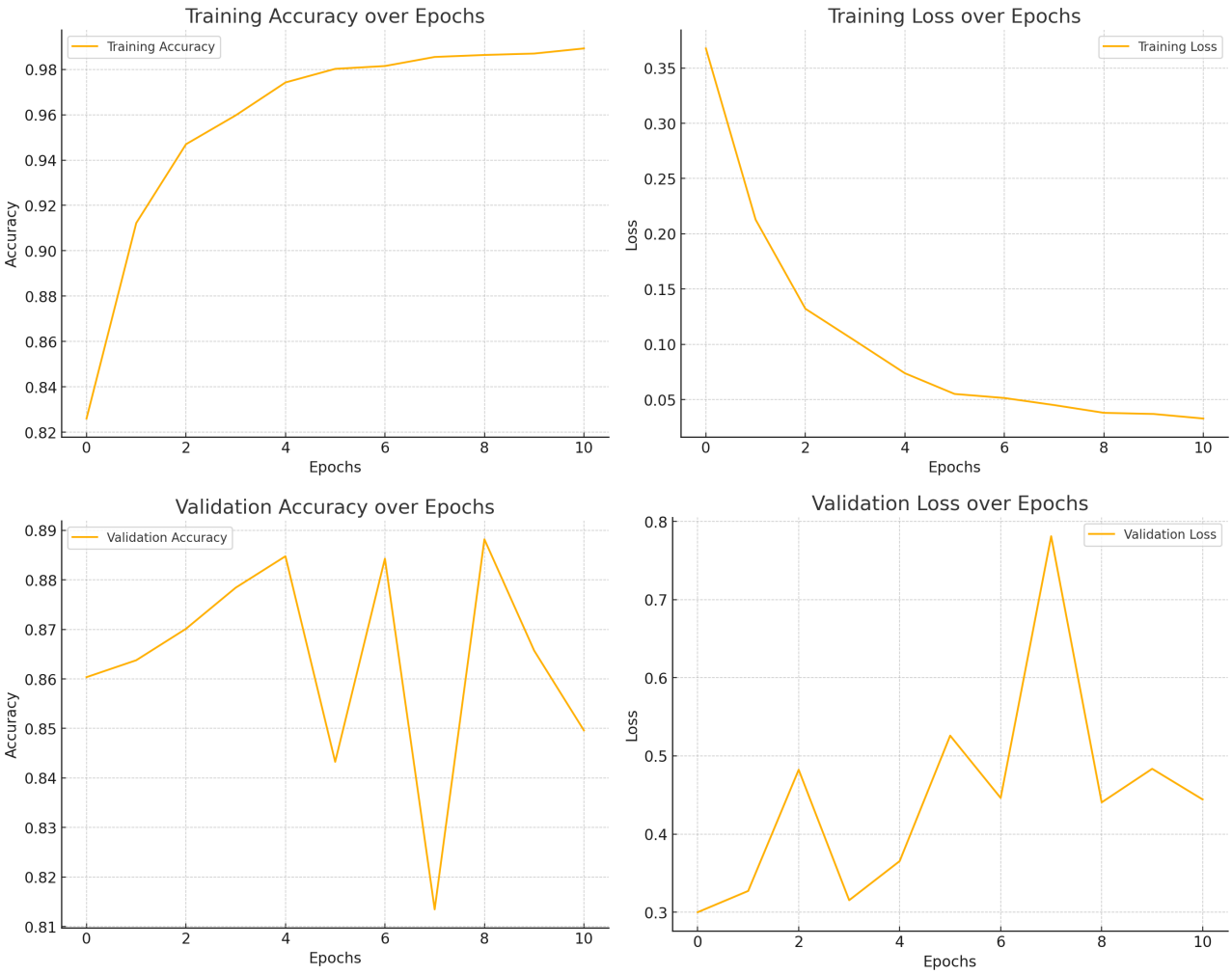


Figure 45: Training and Validation Metrics for Resnet50 with WD regularization (Author, 2024)

Table 4: Metrics for Resnet50 with WD regularization (Author, 2024)

Model	**Dataset size**	**Val Accuracy**	**Test Accuracy**	**Test F1 Score**
ResNet50 AdamW	10,000	0.882	0.903	0.896

5.4.4 Resnet56 - Scaling on full Dataset

Now the best configuration of ResNet50 with weight decay will be scaled to full dataset to increase variety of training data and also to be compared with models trained on subsets. The learning rate finder, used as in previous experiments, identified the optimal learning rate of $6e-4$, Figure 47). The performance of ResNet50, trained with the AdamW optimizer on a dataset of 42,000 samples, demonstrates notable improvements compared to training on smaller subsets. Figure 48 illustrates consistent improvements in both training and validation accuracy and loss over epochs, showcasing stable convergence and generalization. The comparison with models trained on subsampled datasets reveals that scaling the training data size enhances model performance. The confusion matrix (Figure 49) further emphasizes the model's supe-

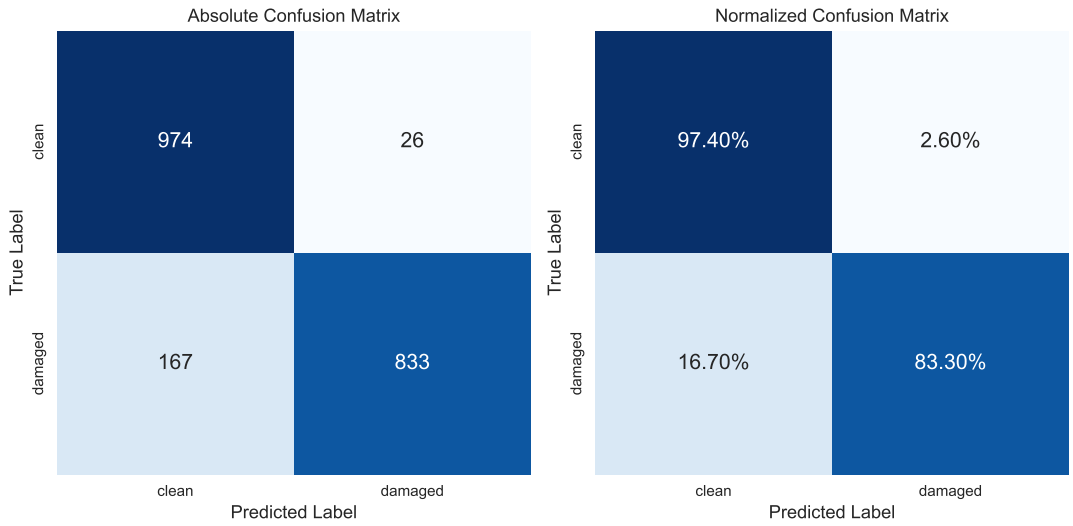


Figure 46: Confusion Matrix for ResNet50 with WD regularization - Test Dataset Performance (Author, 2024)

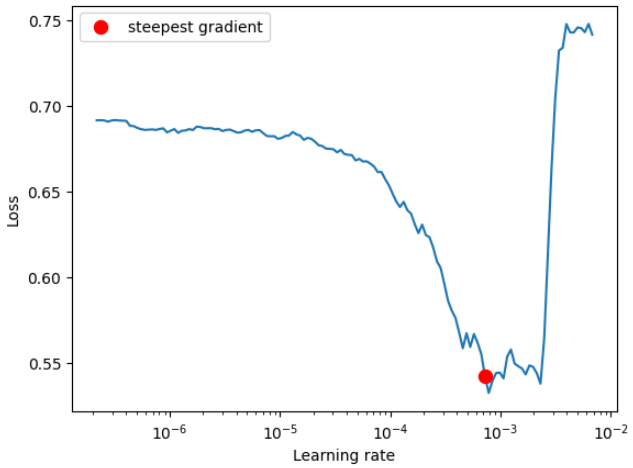


Figure 47: Loss function changes of ResNet50 with AdamW depends on learning rates, highlighting the steepest gradient (Author, 2024)

rior classification capabilities when leveraging the full dataset. Table 5 highlights the impact of training on the larger dataset, with validation accuracy increasing to 91%, test accuracy reaching 93.5%, and the test F1 score improving to 0.936.

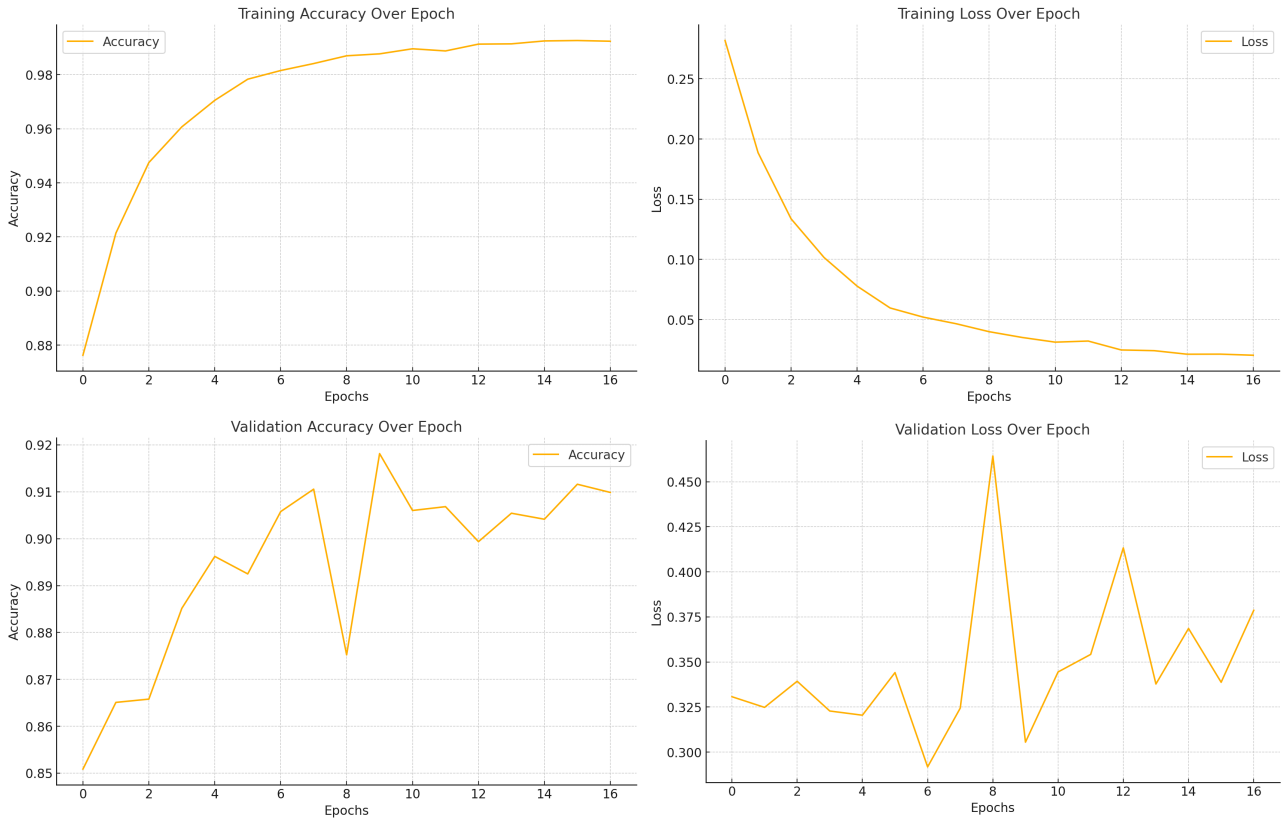


Figure 48: Training and Validation Metrics for Resnet50 with WD regularization - full dataset (Author, 2024)

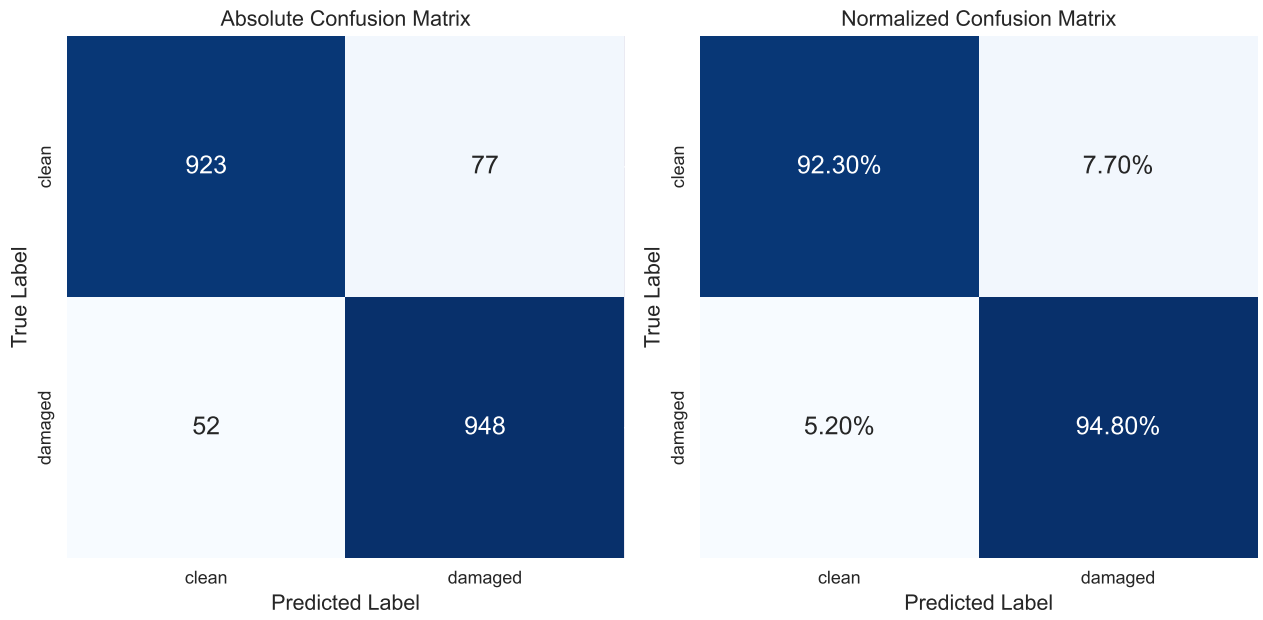


Figure 49: Confusion Matrix for ResNet50 with WD regularization - Test Dataset Performance (Author, 2024)

Table 5: Metrics for Resnet50 with WD regularization (Author, 2024)

Model	Dataset size	Val Accuracy	Test Accuracy	Test F1 Score
ResNet50 AdamW	42,000	0.91	0.935	0.936

The performance of ResNet50, trained with the AdamW optimizer on a dataset of 42,000 samples, demonstrates notable improvements compared to training on smaller subsets. Results for ResNet50 models would be aggregated in Table 9.

5.5 ConvNeXt Tiny

The benchmarking process continues with the ConvNeXt-tiny model, a recent architecture, developed by Facebook (Liu et al., 2022), designed to provide state-of-the-art accuracy performance while maintaining the parameters efficiency. This section replicates the benchmarking methodology used for the ResNet50 model, including both a stock model and variants with regularization.

5.5.1 ConvNeXt Tiny - Stock

The initial benchmarking phase starts with the "stock" ConvNeXt Tiny model. Pre-trained weights are loaded using the Torch library, and the final fully connected layer is modified to handle the binary classification task. Below is the adapted implementation:

```
from torchvision.models import convnext_tiny, ConvNeXt_Tiny_Weights
from torchvision.models.convnext import LayerNorm2d

model = convnext_tiny(weights=ConvNeXt_Tiny_Weights.DEFAULT)
n_inputs = None
for name, child in model.named_children():
    if name == 'classifier':
        for sub_name, sub_child in child.named_children():
            if sub_name == '2':
                # Get the number of input features for the last layer
                n_inputs = sub_child.in_features

sequential_layers_gelu = nn.Sequential(
    LayerNorm2d((n_inputs,), eps=1e-06, elementwise_affine=True),
    nn.Flatten(start_dim=1, end_dim=-1),
    nn.Linear(n_inputs, 1024),
    nn.GELU(),
    nn.Linear(1024, 2),
)
model.classifier = sequential_layers_gelu  # assign a modified classifier
```

Listing 15: Importing the ConvNeXt-tiny model from torch repository and modifying model's classifier (Author, 2024)

The more complex design of the classifier was dictated by the default ConvNeXt's classifier. The learning rate optimization is performed using the LRFinder package to identify the optimal initial learning rate for training, similar to the previous experiment:

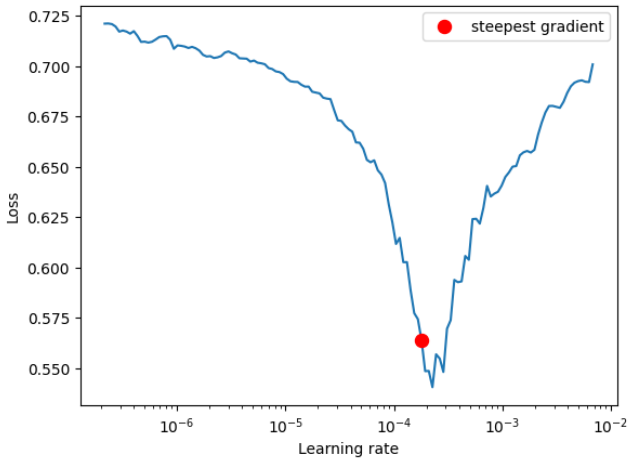


Figure 50: Loss function changes of ConvNeXt-tiny depends on learning rates, highlighting the steepest gradient (Author, 2024)

The steepest gradient was identified at a learning rate of $1.80e-4$. This value was selected as the base learning rate for training. The stock ConvNeXt-tiny model was trained using the following configuration: an 80:20 train-to-validation split, a batch size of 64, the CosineAnnealingLR scheduler, and an early stopping patience of 10 epochs. The script below initiates the training process for the ConvNeXt-tiny model using the specified configuration parameters.

```
loss_func = nn.CrossEntropyLoss()
opt = optim.Adam(model.parameters(), lr=LR)
scheduler = optim.lr_scheduler.CosineAnnealingLR(opt, T_max=EPOCHS)
early_stopper = EarlyStopper(patience=10, min_delta=0.01, verbose=True)

trained_model = train(model, opt,
                      loss_func,
                      train_dataloader,
                      val_dataloader,
                      EPOCHS,
                      experiment_name=EXPERIMENT_NAME,
                      schedule=scheduler,
                      early_stopper=early_stopper)
```

Listing 16: Configuration for the ConvNeXt-tiny model (Author, 2024)

The training and validation performance metrics for the ConvNeXt-tiny model are illustrated in Figure 51.

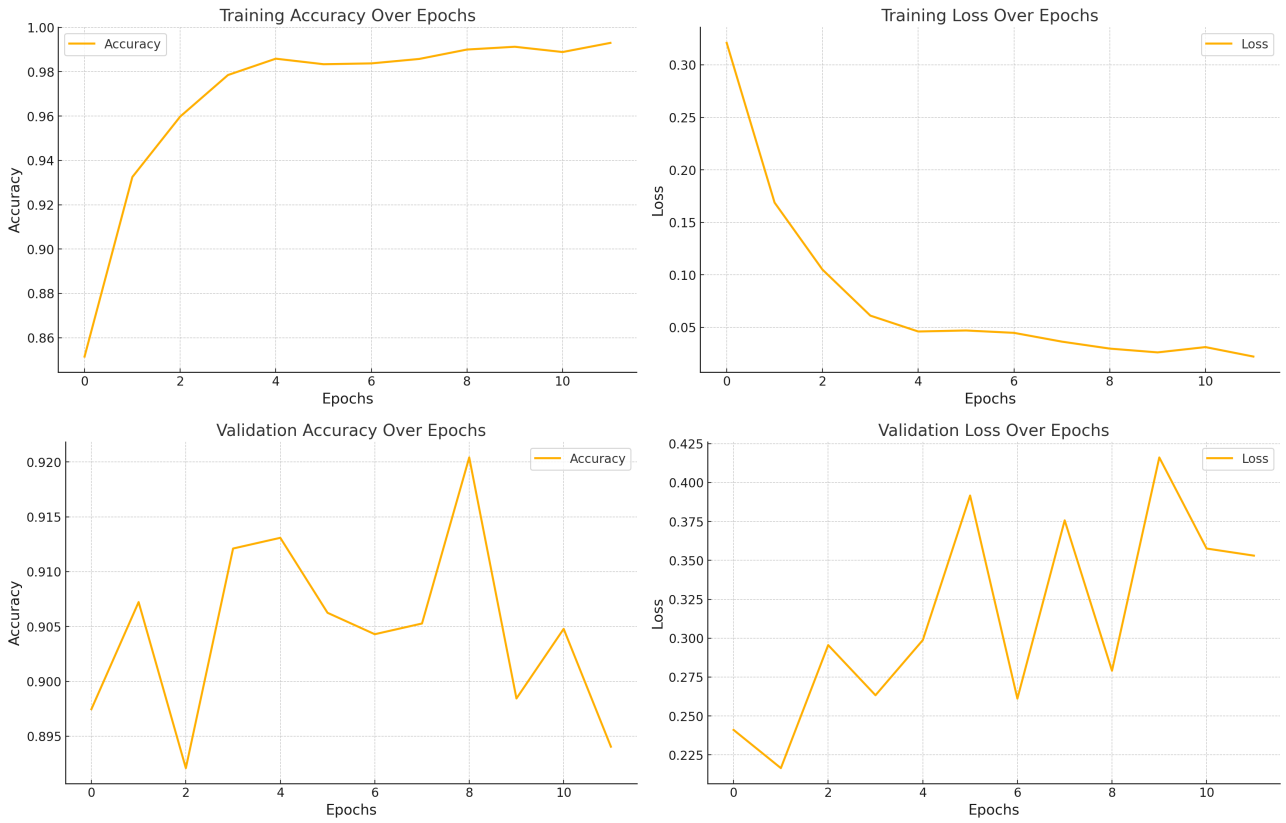


Figure 51: Training and Validation Metrics for ConvNeXt-tiny without regularization (Author, 2024)

The model's performance was further evaluated using an independent test dataset, generating a confusion matrix Figure 52 and key performance metrics, as summarized in Table 6. The ConvNeXt-tiny model achieved a test accuracy of 92.2% and an F1 score of 92.4%, outperforming the stock ResNet50 model, which achieved only a test accuracy of 88.6% and an F1 score of 88.2% (Table 2).

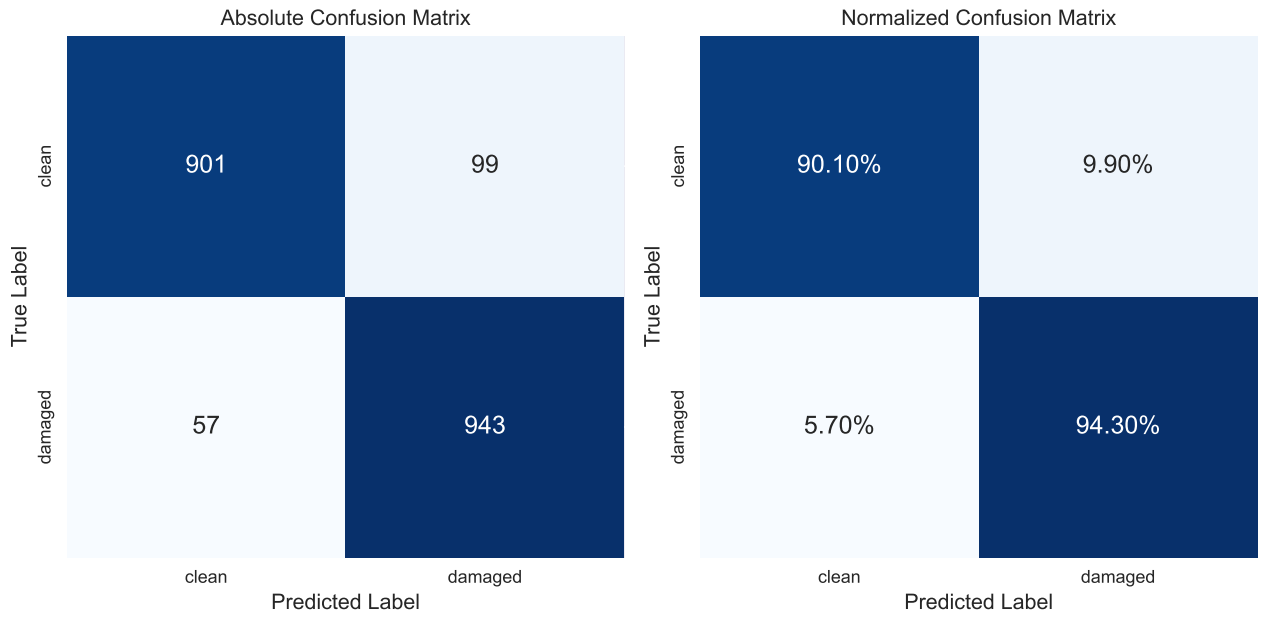


Figure 52: Confusion Matrix for stock ConvNeXt-tiny - Test Dataset Performance (Author, 2024)

Table 6: Metrics for stock ConvNeXt-tiny (Author, 2024)

Model	**Dataset size**	**Val Accuracy**	**Test Accuracy**	**Test F1 Score**
ConvNeXt-tiny	10,000	0.92	0.922	0.924

5.5.2 ConvNeXt Tiny - AdamW

The next step involves modifying the optimizer to AdamW and evaluating the impact of regularization. For this experiment, a weight decay of $1e-2$ was used, with a batch size of 64 and the CosineAnnealingLR. Early stopping was implemented with a patience value of 10 epochs. To determine the optimal learning rate, a learning rate range test was performed, as shown in Figure 53. The steepest gradient in the loss function was identified at a learning rate of $2.5e-4$, which was selected as the base learning rate for training.

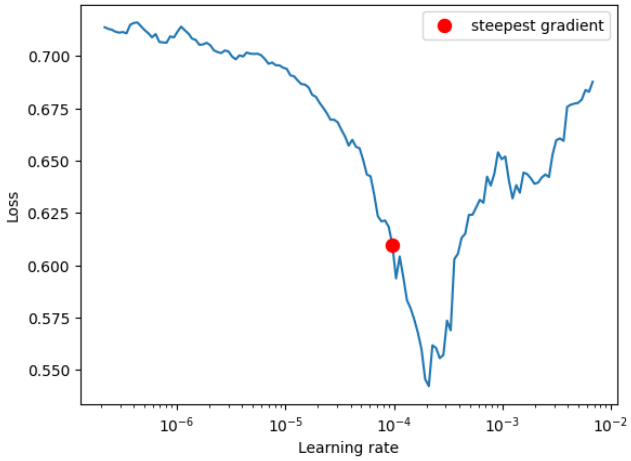


Figure 53: Loss function changes of ConvNeXt-tiny with AdamW depends on learning rates, highlighting the steepest gradient (Author, 2024)

The training and validation performance metrics for the ConvNeXt Tiny model with AdamW are illustrated in Figure 54.

Figure 54: Training and Validation Metrics for ConvNeXt-tiny with WD regularization (Author, 2024)

The model's performance on the independent test dataset is summarized in Table 7. It achieved a test accuracy of 92.7% and an F1 score of 92.6%. The confusion matrix Figure 55 further confirmed it.

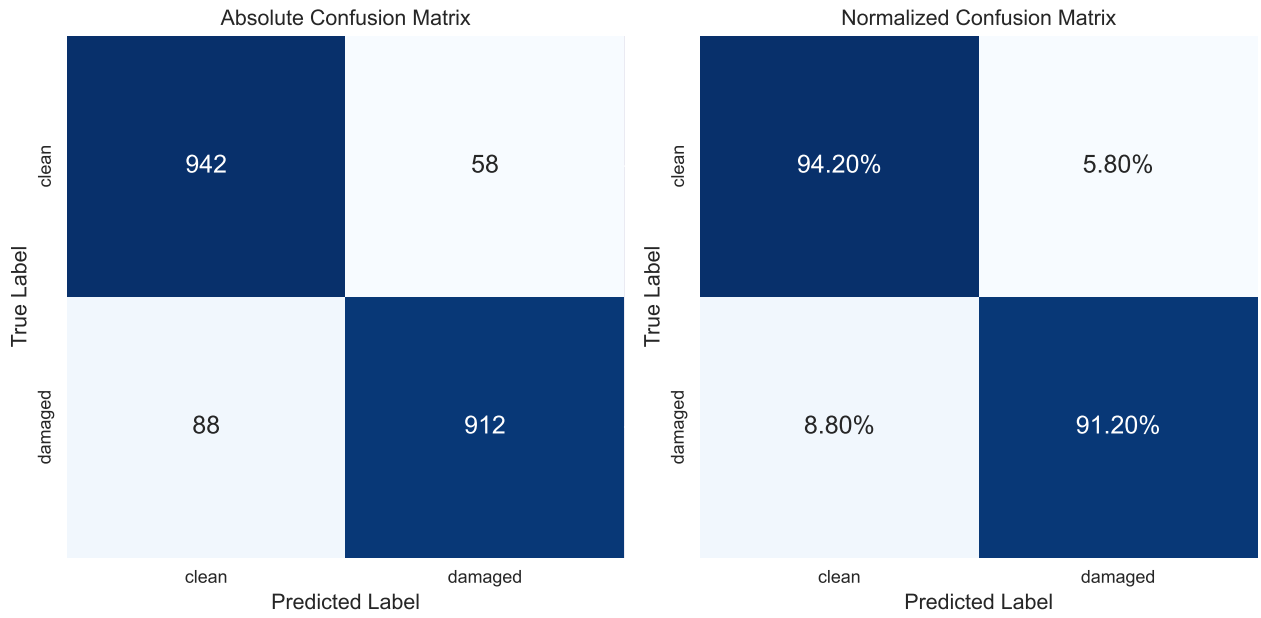


Figure 55: Confusion Matrix for ConvNeXt-tiny with WD - Test Dataset Performance (Author, 2024)

Table 7: Metrics for ConvNeXt-tiny with WD (Author, 2024)

Model	**Dataset size**	**Val Accuracy**	**Test Accuracy**	**Test F1 Score**
ConvNeXt-tiny AdamW	10,000	0.90	0.927	0.926

Compared to the stock ConvNeXt-tiny model, the use of AdamW and regularization enhanced the test accuracy and F1 score. Nevertheless, these changes are within the margin of error.

5.5.3 ConvNeXt Tiny - Scaling on full dataset

Finally, the ConvNeXt Tiny model was trained on the full dataset of 42,000 samples using AdamW as the optimizer. The learning rate was determined using a learning rate finder (Figure 56), with the optimal value identified as $2.5e-4$. Training on the larger dataset further reduced overfitting, as evident from the normalized confusion matrix (Figure 58) and the performance metrics summarized in Table 8.

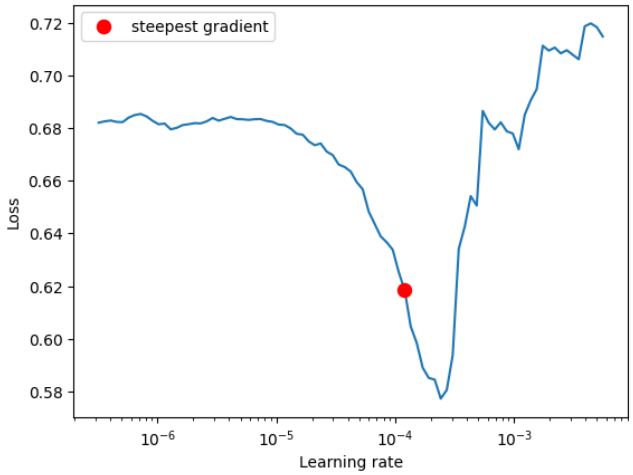


Figure 56: The steepest gradient and suggested LR for ConvNeXt-tiny with AdamW on full-size dataset (Author, 2024)

Training and validation performance metrics are shown in Figure 54.

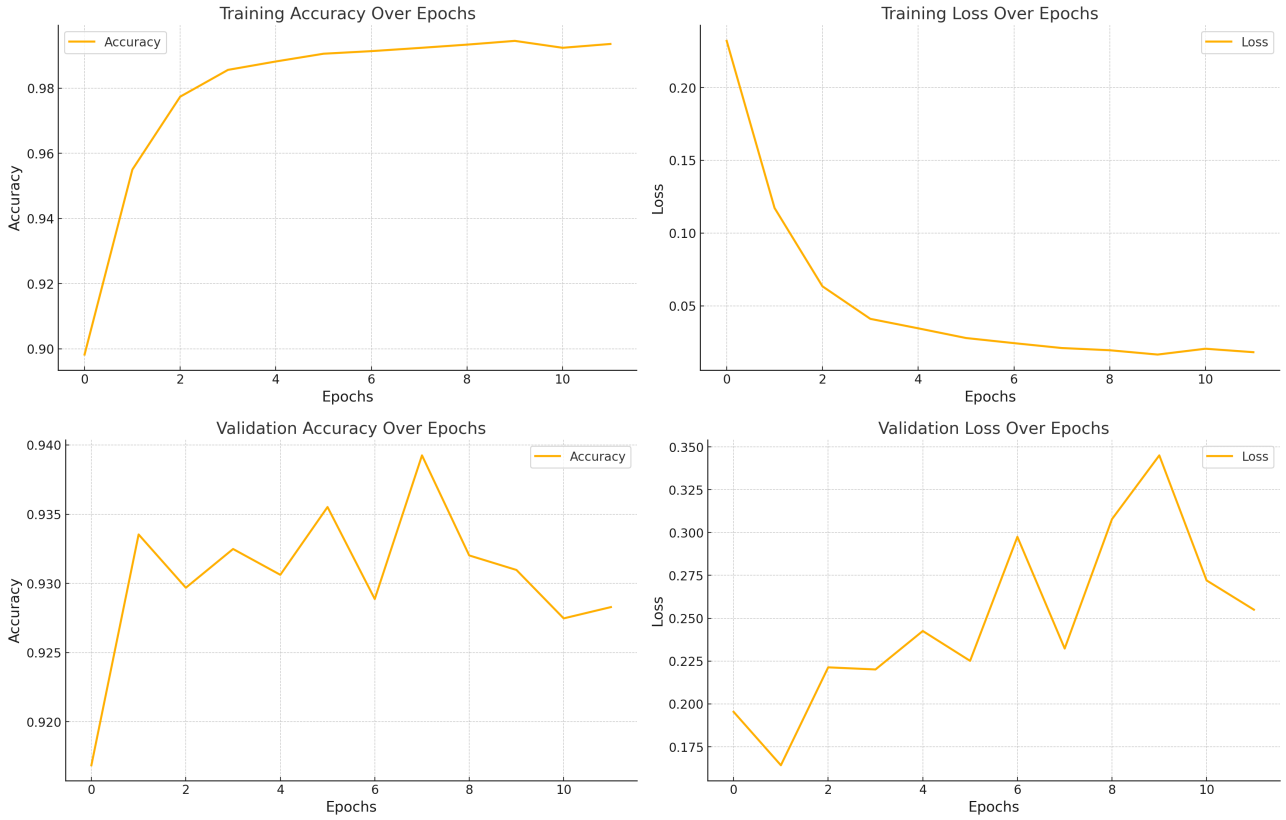


Figure 57: ConvNeXt-tiny - stock version without regularization (Author, 2024)

The confusion matrix in Figure 58 further validates the model's effectiveness on the independent test dataset. The final metrics for the ConvNeXt Tiny model, including validation accuracy, test accuracy, and test F1-score, are summarized in Table 8.

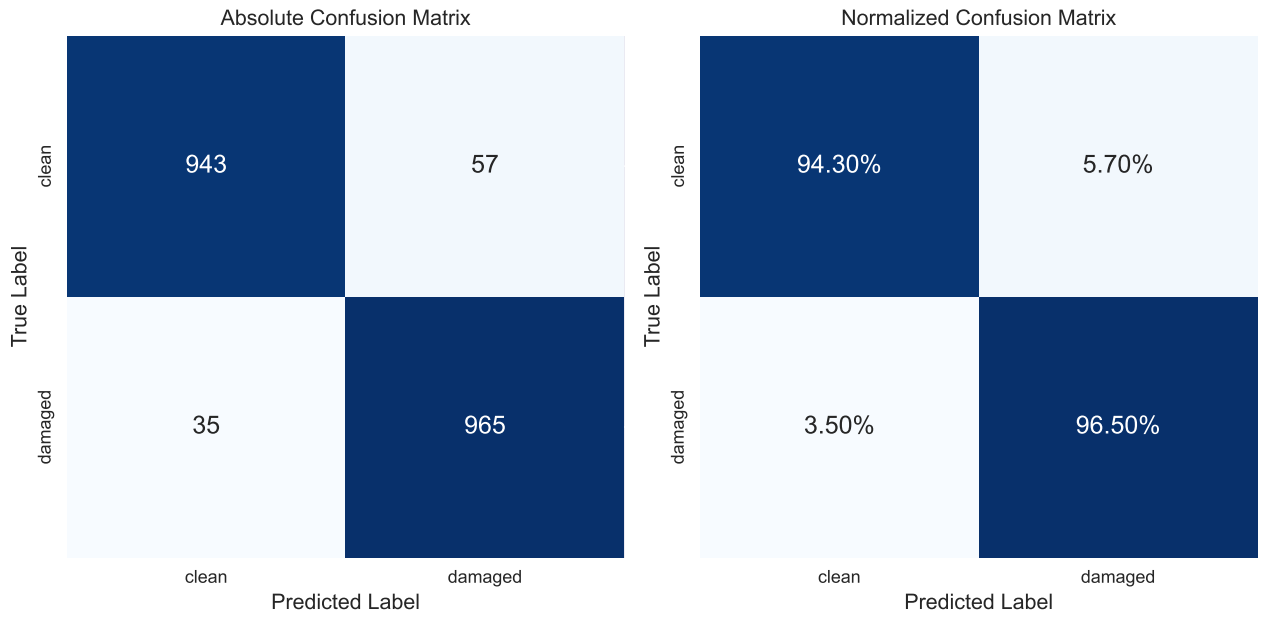


Figure 58: Confusion Matrix for ConvNeXt-tiny with WD regularization (Author, 2024)

Table 8: Metrics for ConvNeXt-tiny with WD regularization on full-size dataset (Author, 2024)

Model	**Dataset size**	**Val Accuracy**	**Test Accuracy**	**Test F1 Score**
ConvNeXt-tiny AdamW	42,000	0.94	0.954	0.955

Among all the models trained during this study, the ConvNeXt Tiny model, trained on full dataset, achieved the best results, both in terms of accuracy and F1-score. Results for ConvNeXt-tiny models would be aggregated in Table 10.

6 Models performance review

6.1 Results & Aggregated Metrics

The performance of the ResNet50 and ConvNeXt models was evaluated using various regularization techniques and dataset sizes, as summarized in Tables 9 and 10. ResNet50 showed consistent improvements when combined with AdamW and dropout, achieving a test accuracy of 93.5% and an F1 score of 0.936 when trained on the full dataset of 42,000 images. ConvNeXt, leveraging its state-of-the-art architecture, achieved a test accuracy of 95.4% and an F1 score of 0.955, making it the top-performing model across all configurations.

Table 9: Metrics for Resnet50 with different regularization and dataset sizes (Author, 2024)

Model	Dataset size	Val Accuracy	Test Accuracy	Test F1 Score
ResNet50	10,000	0.851	0.886	0.882
ResNet50 Dropout	10,000	0.853	0.890	0.887
ResNet50 AdamW	10,000	0.882	0.903	0.896
ResNet50 AdamW	42,000	0.91	0.935	0.936

Table 10: Metrics for ConvNeXt-tiny with different regularization and dataset sizes (Author, 2024)

Model	Dataset size	Val Accuracy	Test Accuracy	Test F1 Score
ConvNeXt-tiny	10,000	0.92	0.922	0.924
ConvNeXt-tiny AdamW	10,000	0.90	0.927	0.926
ConvNeXt-tiny AdamW	42,000	0.94	0.954	0.955

6.2 Production Usage & Live Demo

The developed models were benchmarked in a practical labeling task involving 1,000 car photos. In addition to inference speed, the size of the final models was evaluated as a critical factor for deployment scenarios and use-cases. Both ConvNeXt and ResNet50 can be considered lightweight models, making them suitable for real-world production environments. Table 11 presents the results of the benchmark. ConvNeXt processes the batch slightly faster, completing the task in 8.12 seconds (123 images per second) compared to ResNet50's 9.56 seconds (104 images per second). Whereas ConvNeXt has a slightly larger model size of 109.20 MB compared to ResNet50's 90.0 MB, its speed advantage positions it as the better option for production environments, especially in applications where inference speed is critical, like night data-processing pipelines.

Table 11: Technical Benchmark in labeling task for ConvNeXt and ResNet50 models (Author, 2024)

Model	**1,000 images**	**Inference Speed**	**Size**
ConvNeXt	8.12 sec	123 img/s	109.20 MB
ResNet50	9.56 sec	104 img/s	90.0 MB

The ConvNeXt model was subsequently deployed to a VPS to demonstrate its practical application, available on https://eucalytics.uk/. A web-based interface was developed, allowing users to upload car images and receive predictions indicating whether the cars are clean or damaged. Figures 59 and 60 provide screenshots from the live demonstration, showcasing the system's functionality and real-time inference capability.

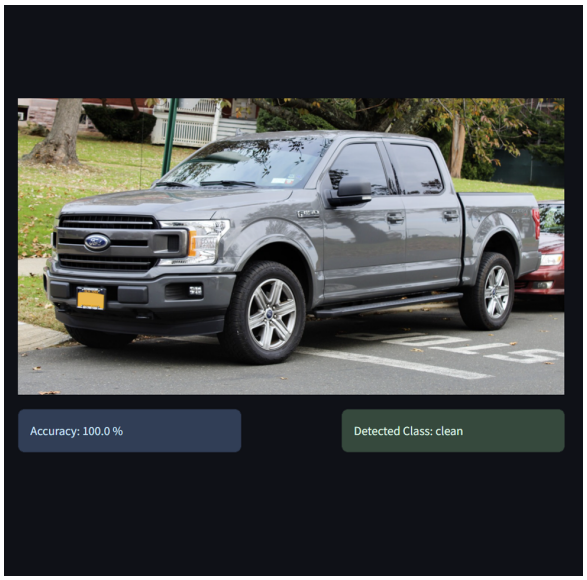


Figure 59: Clean (Author, 2024)

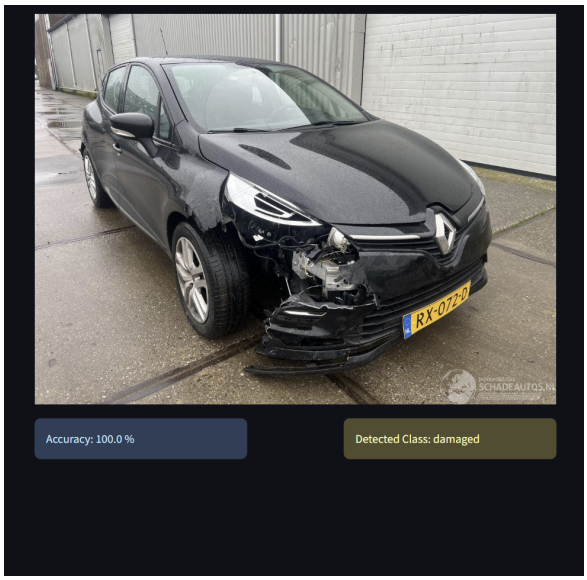


Figure 60: Damaged (Author, 2024)

7 Conclusion

This study demonstrates the application of state-of-the-art deep learning techniques to the task of vehicle damage classification. By leveraging pre-trained convolutional neural networks such as ResNet and ConvNeXt. A custom dataset was curated, refined through a rigorous labeling process, and standardized for optimal usage.

The comparison of different architectures revealed that ConvNeXt achieved the best performance, with a maximum accuracy of 95.4%, underscoring its suitability for this binary classification problem. The analysis also explored key hyperparameter tuning strategies, including learning rate scheduling, finding an area of optimal learning rate with lr-finder module, weight decay regularization, and dropout, which were instrumental in enhancing the model's generalization capability. Furthermore, the incorporation of modern optimization techniques like Adam (and AdamW) demonstrated the benefits of decoupled weight decay in improving model stability and convergence.

A significant contribution of this research is the practical deployment of the final model through a Streamlit-based (Streamlit, n.d.) web application. This interactive tool allows users to upload and classify images of vehicles in real time, bridging the gap between academic research and practical application. The app, supported by a FastAPI (Ramírez, n.d.) back-end and deployed on a virtual private server, provides a user-friendly interface and highlights the potential of this solution for real-world use cases, such as online vehicle marketplaces, auctions, and fraud detection.

This work not only establishes a robust framework for vehicle damage classification but also underscores the importance of data quality and practical deployment. Additionally, the research incorporated metrics collected from an independent test dataset. Even though, there is a noticeable correlation between validation and test accuracy, it is crucial to finally evaluate the models just based on the test sample. This is because even a marginal improvement in validation accuracy does not guarantee consistent performance on test data, and the test metrics may exhibit significant differences.

The methodologies and tools presented in this study are adaptable to similar classification problems, providing a foundation for future advancements in computer vision applications. Future research could explore the integration of Vision Transformers or larger CNN architectures to further enhance performance. Incorporating a third dataset specifically focused on light damages could improve the overall classification accuracy by addressing uncertain cases. Additionally, adding data augmentation techniques, such as gray-scale transformations, could help mitigate the effects of shadows and reflections. Expanding the input context by using models that accept images larger than 224 pixels may also contribute to improved classification by capturing more detailed visual information.

List of Figures

List of Tables

References

Adaloglou, N. (2020, July). *Understanding the receptive field of deep convolutional networks* [Retrieved from AI Summer on July 2, 2020]. https://theaisummer.com/receptive-field/

Andrey Zimovnov. (2018). *Image is a Tensor*. https://github.com/ZEMUSHKA/mml-minor/blob/master/lectures/lecture4.pdf

Araujo, A., Norris, W., & Sim, J. (2019). Computing receptive fields of convolutional neural networks. *Distill*, *4*(11), e21. https://doi.org/10.23915/distill.00021

Compsci 682 neural networks: A modern introduction. (2020). https://compsci682.github.io/notes/neural-networks-1/

Curtis G. Northcutt, M. J., Anish Athalye. (2020). Pervasive label errors in test sets destabilize machine learning benchmarks. *arXiv preprint arXiv:2006.08050*.

David Silva, Nale Raphael. (2020). *PyTorch learning rate finder* [GitHub repository]. https://github.com/davidtvs/pytorch-lr-finder

Fisher, R. A. (1922). On the mathematical foundations of theoretical statistics. *Philosophical Transactions of the Royal Society A*, *222*, 309–368.

Goh, G. (2017). Why momentum really works. *Distill*. https://doi.org/10.23915/distill.00006

Golle, P. (2008). Machine learning attacks against the asirra captcha. *Proceedings of the 15th ACM Conference on Computer and Communications Security*, 535–542. https://doi.org/10.1145/1455770.1455838

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning* [http://www.deeplearningbook.org]. MIT Press.

Hendrycks, D., & Gimpel, K. (2016). Gaussian Error Linear Units (GELUs) [arXiv:1606.08415 [cs]]. https://doi.org/10.48550/arXiv.1606.08415

Ilya, L., & Frank, H. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/pdf/1711.05101.pdf

Ivan. (2024). Zixxatis/photo-convent3r [computer software] [Original work published 2024]. https://github.com/Zixxatis/Photo-Convent3R

Jermey Jordan. (2018). *Setting the learning rate of your neural network*. https://www.jeremyjordan.me/nn-learning-rate/

Kingma, D. P., & Ba, J. (2017). Adam: A method for stochastic optimization [arXiv:1412.6980 [cs.LG]]. *arXiv*, *arXiv:1412.6980*, 6–7. https://doi.org/10.48550/arXiv.1412.6980

Krause, J., Stark, M., Deng, J., & Fei-Fei, L. (2013). 3d object representations for fine-grained categorization. *2013 IEEE International Conference on Computer Vision Workshops*, 554–561. https://doi.org/10.1109/ICCVW.2013.77

Kyu, P. M., & Woraratpanya, K. (2020). Car Damage Detection and Classification. *Proceedings of the 11th International Conference on Advances in Information Technology*, 1–6. https://doi.org/10.1145/3406601.3406651

Li, H., Xu, Z., Taylor, G., Studer, C., & Goldstein, T. (2018a). Visualizing the loss landscape of neural nets. *Neural Information Processing Systems*.

Li, H., Xu, Z., Taylor, G., Studer, C., & Goldstein, T. (2018b). Visualizing the loss landscape of neural nets [arXiv:1712.09913 [cs.LG]]. *arXiv*, *arXiv:1712.09913*. https://doi.org/10.48550/arXiv.1712.09913

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). A ConvNet for the 2020s [ISSN: 2575-7075]. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 11966–11976. https://doi.org/10.1109/CVPR52688.2022.01167

OpenAI. (2024). Chatgpt-4o [large language model] [https://chat.openai.com/chat].

PyTorch Vision Team. (n.d.-a). Pytorch documentation [Retrieved from https://pytorch.org/docs/stable/generated/torch.optim.Adam.html].

PyTorch Vision Team. (n.d.-b). Pytorch documentation [Retrieved from https://pytorch.org/docs/stable/generated/torch.nn.Dropout.html].

PyTorch Vision Team. (n.d.-c). Table of all available classification weights. https://pytorch.org/vision/main/models.html#table-of-all-available-classification-weights

Ramírez, S. (n.d.). *Fastapi* [Repository: https://github.com/fastapi/fastapi]. https://fastapi.tiangolo.com

Ratan, P. (2020, October). What is the convolutional neural network architecture? [Accessed: 2024-12-03].

Smith, L. N. (2017). Cyclical learning rates for training neural networks. *arXiv preprint arXiv:1506.01186*.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, *15*(1), 1929–1958.

Stanford University. (2017). Cs231n: Deep learning for computer vision. https://cs231n.stanford.edu/

Streamlit. (n.d.). *Streamlit* [Repository: https://github.com/streamlit/streamlit]. https://streamlit.io

PyTorch Vision Team. (2021). Torchvision documentation [Retrieved from https://pytorch.org/vision/main/models/generated/torchvision.models.convnext_small.html#torchvision.models.ConvNeXt_Small_Weights].

Ulyankin, F. (2023). FUlyankin/deep_learning_pytorch: Deep learning with pytorch (Russian). https://github.com/FUlyankin/deep_learning_pytorch/tree/main

Ünal, Y., Öztürk, Ş., Dudak, M. N., & Ekici, M. (2022). Comparison of Current Convolutional Neural Network Architectures for Classification of Damaged and Undamaged Cars. In L. Troiano, A. Vaccaro, R. Tagliaferri, N. Kesswani, I. Díaz Rodriguez, I. Brigui, & D. Parente (Eds.), *Advances in Deep Learning, Artificial Intelligence and Robotics* (pp. 141–149). Springer International Publishing. https://doi.org/10.1007/978-3-030-85365-5_14

Vaiana, L. A. (2020). *Applications of artificial intelligence and neural networks to automatic detection of defects on car bodies* [Laurea Thesis]. Politecnico di Torino [Available online at Politecnico di Torino's digital library.]. https://webthesis.biblio.polito.it/16639/

VisionLab, S. (2009). Imagenet [Retrieved from https://www.image-net.org/download.php].

Wager, S., Wang, S., & Liang, P. (2013). Dropout training as adaptive regularization [Available at http://arxiv.org/abs/1307.1493]. *arXiv*, *arXiv:1307.1493*. http://arxiv.org/abs/1307.1493

Weights & Biases. (2020). Relu vs sigmoid function in deep neural networks. https://wandb.ai/ayush-thakur/dl-question-bank/reports/ReLU-vs-Sigmoid-Function-in-Deep-Neural-Networks--VmlldzoyMDk0MzI

Wilber, J. (n.d.). Train, test, and validation sets [Retrieved from https://mlu-explain.github.io/train-test-validation].